Customer Journey Path Optimization

Final Report

by

Aye Khin Khin Hpone | Yosakorn Sirisoot

(Yolanda Lim) 125970 | 126512

AT70.24 – Algorithms Design and Analysis

Instructor: Dr. Chantri Polprasert

Teaching Assistant: Puskar Adhikari

Asian Institute of Technology

School of Engineering and Technology

Thailand

April 2026

ABSTRACT

Optimizing customer conversion paths is a central problem in computational marketing, where user interactions are modeled as a directed graph and the objective is to identify the highest-probability route from entry to conversion. As graph scale increases, exhaustive search becomes intractable. This work proposes a principled pruning framework for log-probability graphs, providing a formal characterization of the trade-off between search efficiency and path reachability.

A **Probability-Pruned Dijkstra** variant is proposed that introduces a threshold parameter τ to discard low-probability partial paths during traversal, reducing the effective edge set from $|E|$ to $|E'| \leq |E|$ with guaranteed convergence to the baseline as $\tau \to 0$. Five formal correctness theorems are established, including an all-or-nothing property: the pruned algorithm either returns the globally optimal path or reports no path at all.

Evaluation spans 3,000 configurations—2,160 synthetic (graphs up to 10,000 nodes) and 840 real-data rows from RetailRocket and RecSys 2015—under fixed and adaptive τ regimes. Across all 706 path-found rows the optimality gap is exactly 0.0000%, confirming the all-or-nothing guarantee. When the pruned algorithm successfully recovers a path, wall-clock speedups range from 1.6× to 7.2× on synthetic data and up to 5.8× on real-world graphs. When the threshold prunes the optimal path entirely, the algorithm detects infeasibility orders of magnitude faster than the baseline (up to 738× on synthetic data and 18,882× on real-world item-level graphs). On large sparse real-data graphs, an adaptive-τ regime raises path-found rates from near 0% to over 80%, demonstrating that threshold-based pruning provides both fast infeasibility detection and correct path recovery with guaranteed optimality when a path is returned.

CONTENTS

ABSTRACT **ii**

1 INTRODUCTION **1**

1.1 Background of the Study 1

1.2 Statement of the Problem 1

1.3 Objectives 2

1.4 Research Hypothesis 3

1.5 Scope and Applicability 4

2 RELATED WORK AND BACKGROUND **5**

2.1 Algorithmic Foundations of Shortest-Path Search 5

2.2 Heuristic vs. Threshold-Based Approximation 5

2.3 Probabilistic Navigation and Markovian Assumptions 6

2.4 Alternative Computational Paradigms 6

2.5 Structural Characteristics: Cycles and Sparsity 7

2.6 Background on Evaluation Data 7

2.7 Research Gap and Project Scope 7

3 ALGORITHM DESIGN, DATA STRUCTURES, AND IMPLEMENTATION **9**

3.1 System Overview 9

3.2 Problem Formulation 10

3.2.1 Graph Model and Inputs 10

3.2.2 Probabilistic Path Objective and Outputs 10

3.2.3 Constraints and Assumptions 12

3.3 Data Generation and Preprocessing 12

3.3.1 Synthetic Data Generation 12
3.3.2 Real-World Clickstream Data 14
3.3.3 Preprocessing and Input Formats 14
3.4 Graph Construction 15
3.5 Algorithmic Approaches 15
3.5.1 Baseline Algorithm: Dijkstra's Shortest Path 15
3.5.2 Optimized Algorithm: Probability-Pruned Dijkstra 17
3.5.3 Algorithm Comparison 20
3.5.4 Critical-τ Finder 21
3.5.5 k-Shortest Simple Paths 21
3.6 Data Structures 22
3.6.1 Adjacency List 22
3.6.2 Binary Min-Heap (Priority Queue) 22
3.6.3 Hash Maps (Dictionaries) 22
3.7 Experimental Design 22
3.7.1 Simulation Environment 22
3.7.2 Evaluation Metrics 22
3.7.3 Experimental Parameters 23
3.7.4 Experimental Controls 24

4 CORRECTNESS PROOFS AND VALIDATION 25

4.1 Correctness of the Log-Probability Transformation 25
4.2 Correctness of Baseline Dijkstra 25
4.3 Correctness of Probability-Pruned Dijkstra 26
4.3.1 Convergence at $\tau = 0$ 26
4.3.2 Conditional Optimality for $\tau > 0$ 26
4.3.3 The All-or-Nothing Property 26
4.4 Empirical Validation and Reachability 27

4.4.1 Summary of Results 27
4.5 Separation of Claims and Limitations 28
4.5.1 Proven Theoretical Claims 28
4.5.2 Empirical Observations (Expected Outcomes) 28
4.5.3 Empirically Validated Findings 28
4.5.4 Acknowledged Limitations 29

5 COMPLEXITY ANALYSIS AND EMPIRICAL VALIDATION 30
5.1 Analytical Methodology 30
5.2 Time Complexity 30
5.2.1 Baseline Dijkstra 30
5.2.2 Probability-Pruned Dijkstra 31
5.2.3 Empirical Time Validation 31
5.3 Space Complexity 31
5.3.1 Empirical Memory Validation 32
5.4 Summary of Complexity 32

6 EXPERIMENTAL RESULTS AND ANALYSIS 34
6.1 Synthetic Experiment Results 34
6.1.1 Speedup vs. Pruning Threshold τ 34
6.1.2 Speedup vs. Graph Size $|V|$ 36
6.1.3 Speedup by Graph Type 38
6.1.4 Speedup by Probability Distribution 38
6.1.5 Optimality Gap Analysis 38
6.1.6 Interpreting All-Rows vs. Found-Only Speedups 40
6.2 Real-Data Experiment Results 40
6.2.1 Datasets and Graph Statistics 40
6.2.2 Fixed-τ Speedup Results 41

6.2.3 Adaptive-τ Experiment 41
6.2.4 Reachability Analysis 42
6.3 Summary of Findings 44

7 CONCLUSION AND FUTURE WORK **48**
7.1 Summary of Contributions 48
7.2 Limitations 49
7.3 Future Work 50

8 PROJECT PLAN **51**
8.1 Project Timeline 51
8.2 Milestones 52

REFERENCES **53**

APPENDIX A: SOURCE CODE AVAILABILITY **55**

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

Understanding how users navigate through digital systems—such as e-commerce websites, mobile applications, and customer support platforms—is essential for improving conversion rates, user satisfaction, customer retention, and operational efficiency. User interaction data captures detailed behavioral patterns that reveal how users navigate through different system components. Customer journeys, represented as sequences of interaction states (e.g., *Landing* $\rightarrow$ *Browse* $\rightarrow$ *Product* $\rightarrow$ *Cart* $\rightarrow$ *Checkout*), provide a structured framework for analyzing these behaviors.

Modern digital systems generate millions of user sessions across thousands of interaction states. When aggregated, these journeys form large directed graphs whose scale and complexity necessitate efficient computational analysis grounded in graph theory and data mining.

1.2 Statement of the Problem

As the volume and complexity of user interaction data continue to grow, identifying meaningful and optimal customer journeys becomes increasingly challenging. From a business perspective, customer journey dynamics directly influence conversion rates, revenue generation, customer retention, and user satisfaction. Even small improvements in the probability of reaching a conversion state can result in significant financial gains at scale.

Naïve analytical methods, such as exhaustive path enumeration or simple frequency-based techniques, do not scale. In large graphs, the number of possible simple paths increases combinatorially, rendering exhaustive enumeration computationally infeasible. Furthermore, directly multiplying many small transition probabilities along a path leads to numerical underflow—a form of arithmetic precision loss that makes naïve probability computation unreliable for longer paths.

The key challenge is therefore to identify optimal customer journeys efficiently and with numerical stability at scale. Core tasks include finding highest-probability conversion paths and

analyzing scalability across large graphs. Each task poses distinct challenges in graph traversal, optimization, and algorithm design that require careful data structure selection.

Maximizing path probability is equivalent to optimizing for the most likely conversion sequence. In an e-commerce context, the highest-probability path from a landing page to checkout represents the conversion funnel that customers naturally follow. Identifying this path allows businesses to reduce friction along the dominant route, prioritize UI investments on the states that carry the most traffic, and design nudges that align with observed user behavior rather than assumed behavior. The term "optimization" in the project title therefore refers not only to algorithmic speedup but also to the actionable business insight that optimal-path discovery enables.

1.3 Objectives

The objective of this study is to design and implement an efficient algorithmic framework for customer journey path optimization grounded in graph theory and classical algorithm design principles. Specifically, the project models customer journey data as a directed weighted graph, where nodes represent interaction states and edges represent transition probabilities between states.

The primary focus is to compute optimal conversion paths under probabilistic constraints. To achieve this, transition probabilities are transformed into additive non-negative edge weights using the negative logarithm, enabling the application of Dijkstra's shortest-path algorithm for maximum-probability path computation. This transformation also addresses the numerical underflow problem by converting multiplicative probability products into additive sums. The project implements Dijkstra's algorithm as a baseline approach and proposes an optimized probability-pruned search strategy that reduces the exploration of low-probability paths to improve computational efficiency.

In addition to correctness, the project formally analyzes the theoretical time and space complexity of both the baseline and optimized algorithms. Empirical performance evaluations have been conducted on graphs of varying sizes and densities to examine scalability, runtime behavior, and trade-offs between computational efficiency and path optimality. Through these analyses, the project demonstrates how careful algorithm design and data structure selection influence performance in large-scale navigation graphs.

Contribution. This study presents a **controlled pruning framework** for probabilistic shortest-path computation, grounded in classical algorithm design and supported by comprehensive empirical evaluation. The pruning variant is deterministic, requires no domain-specific heuristic (unlike A* [Hart et al., 1968]), and provably converges to the globally optimal solution as $\tau \to 0$ (unlike rank-based methods such as beam search, which provide no optimality guarantee [Lowerre, 1976, Jurafsky and Martin, 2000]). The principal contributions are:

1. A formal characterization of an *all-or-nothing* correctness property (Theorem 4.5): the algorithm returns either the exact optimal path or reports no path found, with no sub-optimal intermediate results.

2. Identification and empirical characterization of the *critical pruning threshold* τ_c—a narrow operational range in which speedup is maximized while preserving path optimality.

3. A systematic evaluation framework spanning synthetic and real-world datasets, demonstrating scalability to graphs with tens of thousands of nodes while maintaining solution quality.

Novelty and Prior Work. While pruning strategies, heuristic search, and threshold-based filtering are individually well-studied in shortest-path literature [Hart et al., 1968, Pearl, 1984, Cormen et al., 2009], to the best of our knowledge, such integration has not been extensively studied in the context of probabilistic customer journey graphs. Specifically, the combination of (i) a deterministic, parameter-controlled pruning mechanism with provable convergence, (ii) a formal all-or-nothing optimality guarantee (Theorem 4.5), and (iii) an adaptive-τ regime empirically validated on real-world e-commerce graphs has not been previously reported. This positions the present work at the intersection of classical graph algorithms and applied clickstream optimization, addressing a gap identified in Chapter 3.

Formally, τ acts as a lower bound on acceptable cumulative path probability, controlling the trade-off between search completeness and computational efficiency.

1.4 Research Hypothesis

The following hypothesis guides the experimental evaluation:

> Probability-Pruned Dijkstra provides meaningful speedup over baseline Dijkstra while preserving path optimality at conservative τ values.

Here, “meaningful speedup” is understood to encompass two operational regimes: rapid infeasibility detection (when the threshold exceeds the optimal-path probability) and genuine path-recovery speedup (when the optimal path survives the threshold). The experimental evaluation characterises both regimes and quantifies the inherent reachability–speed trade-off.

The experimental evidence supporting this hypothesis is presented in Chapter 6; a summary of findings is provided in Chapter 7.

1.5 Scope and Applicability

The framework developed in this study is applicable to domains where user interactions can be modeled as probabilistic directed graphs, including e-commerce conversion funnels, mobile application navigation flows, and customer support routing systems. Beyond industrial applications, the algorithmic contributions—particularly the formal correctness guarantees and the characterization of the reachability–speed trade-off—are relevant to researchers investigating threshold-based pruning strategies in shortest-path computation.

CHAPTER 2

RELATED WORK AND BACKGROUND

2.1 Algorithmic Foundations of Shortest-Path Search

The foundation of this work rests on Dijkstra's algorithm [Dijkstra, 1959], a cornerstone of graph theory for finding the shortest path in non-negative weighted graphs. By maintaining a priority queue to explore nodes in order of increasing accumulated cost, Dijkstra's algorithm guarantees the identification of the globally optimal path. When implemented with a binary min-heap, the algorithm achieves a time complexity of $O(|E|\log|V|)$ [Cormen et al., 2009], which is utilized as our primary baseline.

In the context of customer journeys, we apply a log-probability transformation $w(u,v) = -\log p(u,v)$. This exploits the monotone relationship between probability maximization and additive cost minimization—a standard technique in probabilistic graph optimization [Manning and Schütze, 1999]. Adopting the foundations established by Cormen et al. [2009], our implementation utilizes adjacency lists for $O(|V|+|E|)$ space efficiency and hash maps for $O(1)$ distance lookups, ensuring scalability for the large-scale e-commerce graphs analyzed in Chapter 6.

2.2 Heuristic vs. Threshold-Based Approximation

While Dijkstra's algorithm ensures optimality, its exhaustive nature may lead to unnecessary expansion of low-probability states. Informed search algorithms, notably A* [Hart et al., 1968], utilize an admissible heuristic $h(n)$ to guide the search. However, as noted by Pearl [1984], designing a tight, admissible heuristic for log-probability weights in arbitrary navigation graphs is non-trivial.

Alternative approximations, such as Beam Search (originating in speech recognition [Lowerre, 1976] and widely used in NLP [Jurafsky and Martin, 2000]), limit the search frontier to the top-k candidates. While efficient, Beam Search lacks a formal convergence guarantee and may discard the optimal path entirely. The proposed **Probability-Pruned Dijkstra** variant occupies a middle ground. It is a deterministic, parameter-controlled framework that uses a threshold τ to

prune paths that fall below a minimum probability. Unlike rank-based pruning (Beam Search), this method provides a formal *All-or-Nothing Property*: it either returns the globally optimal path or reports no path at all, ensuring that accuracy is never sacrificed for speed when a result is returned.

2.3 Probabilistic Navigation and Markovian Assumptions

Modeling user interactions as a sequence of state transitions is formally equivalent to a first-order Markov chain [Norris, 1998]. This paradigm has been extensively validated in web navigation and clickstream analysis [Bucklin and Sismeiro, 2003, Liu, 2011]. The transition-based models discussed by Resnick and Varian [1997] serve as the basis for our path-finding approach.

The maximum-probability path problem is related to the Viterbi algorithm [Viterbi, 1967, Rabiner, 1989], which finds the most likely state sequence in Hidden Markov Models. However, while Viterbi is optimized for fixed-length sequences, our framework generalizes the principle to arbitrary directed graphs with cycles and variable path lengths. We assume a First-Order Markov property where the next state depends only on the current state; this assumption is empirically grounded in our analysis of the RetailRocket and YOOCHOOSE datasets, which exhibit stable transition matrices across high-volume interaction sessions. While higher-order dependencies may exist in practice, incorporating them would exponentially increase the state space and is left as future work.

2.4 Alternative Computational Paradigms

Customer journey analysis often intersects with *Sequential Pattern Mining* and *Scalable Graph Processing*. Algorithms like PrefixSpan [Pei et al., 2001] and general surveys of frequent pattern mining [Han et al., 2007] focus on discovering frequent sequences rather than optimizing specific source-to-target queries. While these mining techniques utilize pruning (e.g., minimum support thresholds), they serve a descriptive rather than an optimization-based purpose.

Furthermore, scalable systems like Pregel [Malewicz et al., 2010] and subsequent distributed graph frameworks [Sahu et al., 2017] address the engineering challenges of massive graphs. Our work focuses instead on the *algorithmic characterization* of pruning. By evaluating search-space reduction within a controlled single-machine environment, we establish the fundamental trade-offs between pruning intensity and path reachability that would inform implementations

in distributed environments.

2.5 Structural Characteristics: Cycles and Sparsity

A critical structural property of user navigation is the presence of cycles (e.g., a user returning to a "Browse" state from a "Product" page). Following Bucklin and Sismeiro [2003], we model the journey as a general directed cyclic graph. Because our log-transformed weights are strictly non-negative ($w \geq 0$), Dijkstra's algorithm remains correct and naturally avoids infinite loops, as cycle traversal strictly increases path cost [Cormen et al., 2009].

Additionally, our framework assumes graph sparsity where $|E| \approx O(|V|)$. This assumption is not merely theoretical; it is empirically supported by the datasets used in this study. For instance, the item-level transition graphs derived from RetailRocket exhibit a low average out-degree, reflecting the reality that users typically navigate between a limited set of related products rather than a fully connected network.

2.6 Background on Evaluation Data

To ensure the robustness of the pruning framework, we utilize a dual-pronged evaluation strategy involving both synthetic and real-world data:

- **Synthetic Graphs:** Grounded in the Erdős–Rényi random graph model [Erdős and Rényi, 1960], we implement two generators: a sparse random generator and a layered stage-based generator. These allow for controlled stress-testing of τ across varying node densities and stage-gate structures.
- **Real-World Datasets:** We utilize the **RetailRocket** [Retail Rocket, 2015] and **RecSys 2015 YOOCHOOSE** [Ben-Shimon et al., 2015] datasets. These provide millions of clickstream events to validate the algorithm on actual user behavioral patterns, moving the evaluation beyond purely theoretical or synthetic bounds.

2.7 Research Gap and Project Scope

Despite the maturity of shortest-path theory and clickstream modeling, a gap exists in the systematic, empirical evaluation of *threshold-based pruning* specifically for cyclic customer journey graphs. Existing literature tends to treat sequential mining [Pei et al., 2001] and shortest-path search [Dijkstra, 1959] as separate domains.

This project addresses this gap by presenting a controlled pruning framework that integrates probabilistic transition modeling with formal optimality characterizations. Rather than proposing a fundamentally new shortest-path theory, this work focuses on the empirical characterization of the time–optimality trade-off, providing a principled method for reducing the search space in high-scale conversion optimization problems while maintaining a mathematical guarantee of path optimality.

CHAPTER 3

ALGORITHM DESIGN, DATA STRUCTURES, AND IMPLEMENTATION

3.1 System Overview

The *Customer Journey Path Optimization* system models user interactions as a directed weighted graph $G = (V, E)$, where each node $v \in V$ represents a discrete interaction state (e.g., *Landing Page*, *Search*, *Product Page*, *Cart*, *Checkout*), and each directed edge $(u, v) \in E$ represents a user transition between states annotated with a transition probability $p(u, v) \in (0, 1]$. The primary goal is to identify the most probable path from a designated source node s to a target conversion node t.

Figure 3.1 presents the end-to-end workflow. It is organized into five sequential components: (1) data generation and preprocessing, (2) graph construction, (3) baseline path optimization using Dijkstra's algorithm, (4) optimized path search using probability-pruned Dijkstra, and (5) experimental evaluation and comparative analysis.

Two algorithms were implemented and compared throughout this chapter. Algorithm 1 (Baseline Dijkstra on Log-Weights) applies standard Dijkstra's algorithm to the log-probability-transformed graph, guaranteeing globally optimal paths in $O(|E|\log|V|)$ time. Algorithm 2 (Probability-Pruned Dijkstra) extends Algorithm 1 with a pruning threshold τ that discards partial paths whose cumulative probability falls below τ, reducing expected runtime to $O(|E'|\log|V|)$, $|E'| \leq |E|$, with convergence to Algorithm 1 as $\tau \to 0$.

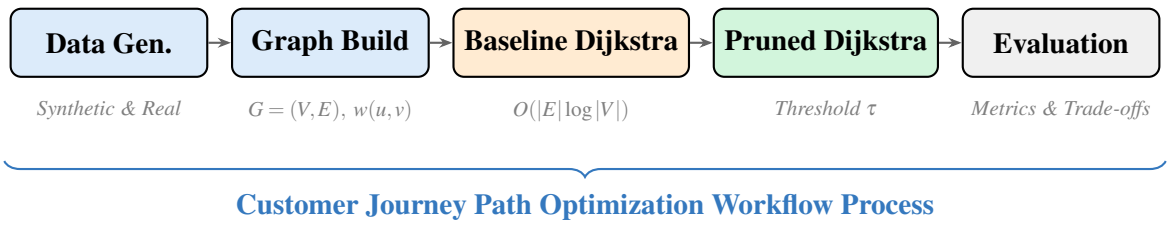

Figure 3.1. End-to-end system workflow.

3.2 Problem Formulation

3.2.1 Graph Model and Inputs

Customer interactions are modeled as a directed graph $G = (V, E)$, where V is the set of interaction states (nodes), $E \subseteq V \times V$ is the set of directed transitions (edges), and each edge $(u, v) \in E$ carries a transition probability $p(u, v) \in (0, 1]$. A designated source node $s \in V$ (e.g., Landing Page) and target node $t \in V$ (e.g., Checkout) define the query endpoints. In the optimized algorithm, a pruning threshold $\tau \in (0, 1]$ is used to discard low-probability partial paths.

Figure 3.2 illustrates a representative five-node customer journey graph. Node abbreviations: LP = Landing Page (source s), SR = Search Results, PP = Product Page, CT = Cart, CK = Checkout (target t). The solid blue path LP $\rightarrow$ SR $\rightarrow$ PP $\rightarrow$ CK is optimal ($\Pi^* = 0.336$, log-cost ≈ 1.091).

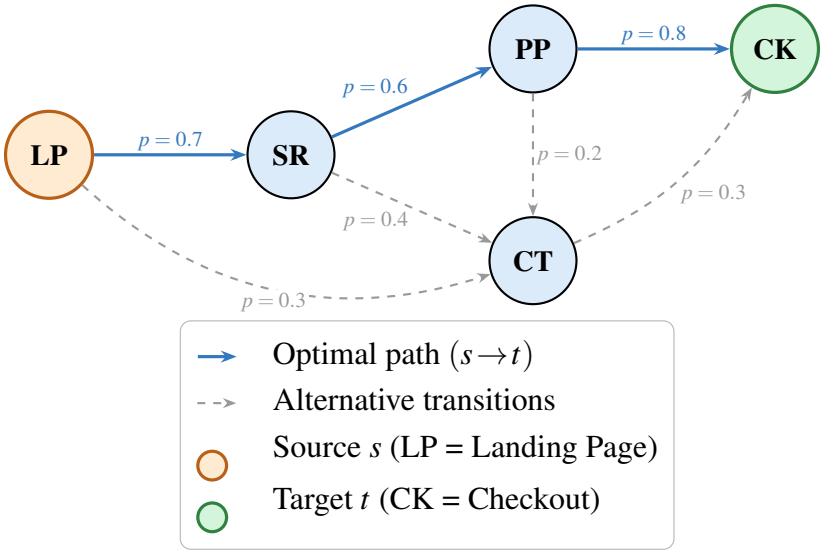

Figure 3.2. Five-node customer journey graph (s = LP, t = CK).

3.2.2 Probabilistic Path Objective and Outputs

The goal is to find a path $P^* = (v_0 = s, v_1, \ldots, v_k = t)$ that maximizes the joint path probability [Norris, 1998]:

$$\max_{P} \prod_{(u,v) \in P} p(u, v)$$

Directly multiplying many small probabilities causes numerical underflow [Manning and Schütze, 1999]. Each transition probability is transformed into an additive non-negative edge

weight using the negative logarithm:

$$w(u,v) = -\log\big(p(u,v)\big)$$

Since $p(u,v) \in (0,1]$, it follows that $w(u,v) \geq 0$, satisfying Dijkstra's non-negativity requirement [Cormen et al., 2009]:

$$\max \prod_{(u,v)\in P^*} p(u,v) \iff \min \sum_{(u,v)\in P^*} -\log p(u,v)$$

For a given source–target query, the system returns the optimal path $P^* = (v_0 = s, v_1, \ldots, v_k = t)$, the corresponding path probability $\Pi^* = \prod_{(u,v)\in P^*} p(u,v) = \exp(-C^*)$, and the log-cost $C^* = \sum_{(u,v)\in P^*} -\log p(u,v)$. Each run also records performance metrics—execution time, peak memory, nodes explored, edges examined, edges relaxed, and maximum priority queue size—as well as the optimality gap $\Delta = |C^*_{\text{baseline}} - C^*_{\text{pruned}}|/C^*_{\text{baseline}} \times 100\%$.

Concrete worked example: RetailRocket event-level funnel. The RetailRocket dataset [Retail Rocket, 2015] yields a compact 3-node graph. Its MLE transition probabilities, computed from 2.7 million events, provide a concrete illustration of the formulation:

Transition	**Probability**	**Log-weight**
view $\rightarrow$ addtocart	$p = 0.041$	$w = -\ln(0.041) = 3.19$
addtocart $\rightarrow$ transaction	$p = 0.226$	$w = -\ln(0.226) = 1.49$
view $\rightarrow$ transaction (direct)	$p = 0.004$	$w = -\ln(0.004) = 5.52$

Two candidate $s \rightarrow t$ paths exist from *view* to *transaction*:

- **Funnel path:** view $\rightarrow$ addtocart $\rightarrow$ transaction, $\Pi = 0.041 \times 0.226 = 0.0093$ (log-cost 4.68).
- **Direct skip:** view $\rightarrow$ transaction, $\Pi = 0.004$ (log-cost 5.52).

Dijkstra selects the *funnel path* because it has lower log-cost ($4.68 < 5.52$), equivalently higher probability ($0.93\% > 0.39\%$). This $2.3\times$ probability advantage quantifies the value of the add-to-cart step: users who add items to their cart are substantially more likely to complete a transaction than those who skip directly.

3.2.3 Constraints and Assumptions

The framework assumes graph sizes up to 50,000 nodes (hardware permitting), with experiments conducted on graphs of up to 10,000 nodes. Graphs are assumed to be sparse ($|E| = O(|V|)$) and stored using an adjacency-list representation requiring $O(|V|+|E|)$ space. All transition probabilities are strictly positive; zero-probability transitions are treated as non-existent edges. The customer journey graph may contain directed cycles; correctness is preserved because $w(u,v) \geq 0$ for all edges. Finally, a first-order **Markov assumption** is adopted: $P(\text{next} \mid \text{current})$ is independent of earlier states. This assumption is standard in clickstream modeling [Bucklin and Sismeiro, 2003] and is empirically motivated by the observation that transition matrices estimated from session data yield stable, reproducible graph structures across independent subsamples (a necessary condition for the first-order model, though not a formal test of whether higher-order dependencies are absent). While higher-order dependencies exist (e.g., users arriving via search may navigate differently from those arriving via advertisement), the first-order model captures the dominant transition structure and keeps the state space tractable at $O(|V|)$ rather than $O(|V|^k)$ for order k.

3.3 Data Generation and Preprocessing

3.3.1 Synthetic Data Generation

Two graph generators were implemented in `data/graph_generator.py`.

The first is an **Erdős–Rényi sparse random generator** [Erdős and Rényi, 1960], which creates $|V|$ nodes and samples directed edges with expected total $\bar{d} \cdot |V|$; transition probabilities are drawn from either a uniform $\mathscr{U}(0.01, 1.0)$ or a power-law ($\alpha = 2$) distribution. This generator produces graphs with uniformly random structure, serving as a controlled baseline for evaluating algorithmic behavior without structural bias.

The second is a **layered (stage-based) generator**, which models the funnel-shaped structure typical of real customer journeys. Nodes are distributed across five ordered stages—Awareness, Interest, Consideration, Intent, and Conversion—reflecting the standard e-commerce conversion funnel. Edges are sampled primarily in the forward direction (up to $+2$ stages ahead), ensuring that the dominant flow proceeds from awareness toward conversion. A configurable `backward_prob` parameter (default 0.15) controls the fraction of backward links, modeling the common user behavior of revisiting earlier stages (e.g., returning from a product page to

browse). Transition probabilities for forward edges use raw weights drawn from $\mathcal{U}(0.3, 0.8)$, while backward edges receive lower raw weights from $\mathcal{U}(0.05, 0.3)$, reflecting the empirical observation that forward progression is more likely than backtracking. These raw weights are then normalized per source node (so that $\sum_v p(u,v) = 1$); consequently, the final transition probabilities no longer follow the original uniform distributions but preserve their relative ordering. This layered design is the more application-relevant of the two generators, as it approximates the stage-wise structure observed in real clickstream data (Section 6.2).

Both generators employ $O(n \cdot d)$ scalable edge sampling rather than $O(n^2)$ adjacency-matrix construction. A BFS-based connectivity guarantee ensures that if no $s \to t$ path exists after random sampling, bridge edges are added; these bridge edges are treated as forward edges and therefore receive raw weights from $\mathcal{U}(0.3, 0.8)$. Outgoing probabilities are normalized per node ($\sum_v p(u,v) = 1$), and deterministic seeding via `hashlib.md5` ensures cross-platform reproducibility.

The following code snippet shows key aspects of the Erdős–Rényi generator:

```
def generate_erdos_renyi_graph(n, avg_degree, distribution, seed,
    ...):
    rng = random.Random(seed)
    graph = {f"node_{i}": {} for i in range(n)}
    # O(n*d) edge sampling -- NOT O(n^2)
    for u in graph:
        num_edges = rng.randint(
            max(0, int(avg_degree) - 2), int(avg_degree) + 2)
        targets = rng.sample(other_nodes, min(num_edges, len(
           other_nodes)))
        for v in targets:
            if distribution == "uniform":
                raw = rng.uniform(0.01, 1.0)
            else:  # power_law: Pareto(alpha=2): x_min * (1-U)^(-1/
               alpha) with alpha=2
                raw = min(0.01 * (1 - rng.random()) ** (-0.5), 1.0)
            graph[u][v] = raw
    _normalize_and_log_transform(graph)
    _ensure_connectivity(graph, source, target)
    return graph
```

Listing 3.1: ER graph generator (`data/graph_generator.py`)

3.3.2 *Real-World Clickstream Data*

Two real-world clickstream datasets were used for validation (Table 3.1):

Dataset	Events	Sessions	Graph Size	Source
RetailRocket (event-level)	2.7M	1.4M	3 nodes, 9 edges	Kaggle [Retail Rocket, 2015]
RetailRocket (item-level)	2.7M	50K	44,711 nodes, 101,528 edges	Kaggle [Retail Rocket, 2015]
RecSys 2015 (YOOCHOOSE)	33M	50K	12,935 nodes, 70,442 edges	Kaggle [Ben-Shimon et al., 2015]

Table 3.1. Real-world clickstream datasets used for validation.

Transition probabilities are estimated using maximum likelihood:

$$\hat{p}(u,v) = \frac{\text{count}(u \to v)}{\sum_{w:\,(u,w)\in E} \text{count}(u \to w)}$$

A consequence of MLE on sparse data is that nodes with very few observed transitions (e.g., items viewed only once or twice) receive high-variance probability estimates. In item-level graphs, where tens of thousands of items may each have only a handful of observations, many edges carry very low estimated probabilities. This contributes to the low path-found rates observed in item-level real-data experiments (Section 6.2): when most edge probabilities are small, even the baseline optimal path has a very low cumulative probability, making it more susceptible to pruning. Smoothing techniques such as Laplace (add-one) correction or Bayesian priors could mitigate this effect but were not applied in order to isolate the algorithmic contribution of pruning.

3.3.3 *Preprocessing and Input Formats*

The preprocessing module (`src/preprocessing.py`) accepts two CSV formats:

1. **Direct transitions** — columns `source` and `target`, one row per observed transition.
2. **Session event streams** — columns `session_id` and `state` (plus `step` or `timestamp` for ordering). Consecutive events within each session are converted to pairwise transitions.

Both formats produce the same transition-count table from which MLE probabilities are computed.

3.4 Graph Construction

The directed weighted graph is constructed from the computed transition probabilities using an **adjacency list** representation. The function receives a nested dictionary mapping each source node to its targets and their probabilities, then converts each probability into a log-weight $w(u,v) = -\log(\hat{p}(u,v))$. This provides $O(|V| + |E|)$ space complexity and efficient $O(\deg(u))$ neighbour iteration [Cormen et al., 2009].

```
def build_weighted_graph(transition_probs):
    graph = defaultdict(list)
    for source, targets in transition_probs.items():
        for target, prob in targets.items():
            if prob <= 0:
                continue
            graph[source].append((target, -math.log(prob)))
            if target not in graph:
                graph[target] = []  # ensure sink nodes exist
    return dict(graph)
```

Listing 3.2: Graph construction (`src/graph_builder.py`)

3.5 Algorithmic Approaches

3.5.1 Baseline Algorithm: Dijkstra's Shortest Path

The baseline applies Dijkstra's algorithm [Dijkstra, 1959] to the log-transformed graph. A **binary min-heap** (`heapq`) serves as the priority queue with **lazy insertion**—when a shorter path to a node is found, a new entry is pushed and stale entries are skipped when popped.

Algorithm 1: Dijkstra on Transformed Probabilities [Dijkstra, 1959, Cormen et al., 2009]

Input: Graph $G = (V, E)$ with weights $w(u, v) = -\log p(u, v)$, source s, target t

Output: Shortest-path distance $dist[t]$, probability $\exp(-dist[t])$, path P^*

Initialise $dist[v] \leftarrow \infty$ for all $v \in V$; $parent[v] \leftarrow$ NIL;
$dist[s] \leftarrow 0$; insert $(0, s)$ into min-heap Q;
while *Q is not empty* **do**
    $(d, u) \leftarrow$ Extract-Min(Q);
    if $u = t$ **then**
        break; `// optimal path reached`
    end
    if $d > dist[u]$ **then**
        continue; `// stale entry (lazy deletion)`
    end
    foreach *edge* (u, v) *with weight* $w = -\log p(u, v)$ **do**
        $edges\_examined \leftarrow edges\_examined + 1$;
        $edges\_relaxed \leftarrow edges\_relaxed + 1$; `// no pruning, so every examined edge is relaxed`
        $new\_dist \leftarrow dist[u] + w$;
        if $new\_dist < dist[v]$ **then**
            $dist[v] \leftarrow new\_dist$; $parent[v] \leftarrow u$;
            Insert (new_dist, v) into Q;
        end
    end
end
return $dist[t]$, $\exp(-dist[t])$, ReconstructPath$(parent, s, t)$;

Time complexity: $O(|E| \log |V|)$ with a binary heap [Cormen et al., 2009].

Space complexity: $O(|V| + |E|)$.

Optimality: Guaranteed, since all $w(u, v) \geq 0$ [Dijkstra, 1959].

Worked Example

Table 3.2 traces Algorithm 1 on the five-node graph (Figure 3.2).

Step #	Extract	LP	SR	PP	CT	CK
Init	—	0	∞	∞	∞	∞
1	LP	0	0.36	∞	1.20	∞
2	SR	0	0.36	0.87	1.20	∞
3	PP	0	0.36	0.87	1.20	1.09
4	**CK✓**	0	0.36	0.87	1.20	**1.09**

Table 3.2. Dijkstra `dist[]` trace ($s = \text{LP}$, $t = \text{CK}$).

The column headers are node abbreviations: LP = Landing Page (source s), SR = Search Results, PP = Product Page, CT = Cart, and CK = Checkout (target t). Each row shows the `dist[]` array after extracting the minimum-cost node listed in the **Extract** column. Values are log-costs $w = -\log p$; ∞ denotes an unreached node. At Step 1, extracting LP relaxes its neighbours SR ($-\log 0.7 \approx 0.36$) and CT ($-\log 0.3 \approx 1.20$). At Step 2, SR is extracted and reaches PP ($0.36 + -\log 0.6 \approx 0.87$). Step 3 extracts PP and discovers CK ($0.87 + -\log 0.8 \approx 1.09$). At Step 4, CK is extracted with cost **1.09**; the ✓ annotation marks the *target-reached* early exit. The bold value **1.09** is the final optimal log-cost, corresponding to path probability $\Pi^* = \exp(-1.09) \approx 0.336$.

3.5.2 Optimized Algorithm: Probability-Pruned Dijkstra

The pruned algorithm extends the baseline by introducing a **probability pruning threshold** $\tau \in (0, 1]$. Any partial path whose cumulative probability falls below τ is pruned. In log-space:

$$\text{new\_cost} > T = -\log(\tau) \implies \text{skip edge}$$

Algorithm 2: Probability-Pruned Dijkstra

Input: Graph $G = (V, E)$, source s, target t, pruning threshold $\tau \in (0, 1]$

Output: Shortest-path distance $dist[t]$, probability $\exp(-dist[t])$, path P^*

Initialise $dist[v] \leftarrow \infty$ for all $v \in V$; $parent[v] \leftarrow$ NIL;
$dist[s] \leftarrow 0$; insert $(0, s)$ into min-heap Q;
$T \leftarrow -\log(\tau)$; `// pruning threshold in log-space`
while *Q is not empty* **do**
    $(d, u) \leftarrow$ Extract-Min(Q);
    if $u = t$ **then**
        break
    end
    if $d > dist[u]$ **then**
        continue; `// stale entry`
    end
    foreach *edge* (u, v) *with weight* $w = -\log p(u, v)$ **do**
        $edges\_examined \leftarrow edges\_examined + 1$;
        $new\_dist \leftarrow dist[u] + w$;
        if $new\_dist > T$ **then**
            continue; `// PRUNE: below` τ
        end
        $edges\_relaxed \leftarrow edges\_relaxed + 1$;
        if $new\_dist < dist[v]$ **then**
            $dist[v] \leftarrow new\_dist$; $parent[v] \leftarrow u$;
            Insert (new_dist, v) into Q;
        end
    end
end
return $dist[t]$, $\exp(-dist[t])$, ReconstructPath$(parent, s, t)$;

Worst-case time complexity: $O(|E| \log |V|)$ (when $\tau \to 0$).

Empirically observed time complexity: $O(|E'| \log |V|)$, $|E'| \leq |E|$ (see Chapter 5 for validation).

Space complexity: $O(|V| + |E|)$.

The `edges_relaxed` Metric

The `edges_relaxed` counter has a precise, intentional definition that differs between the two algorithms:

Algorithm	Metric	Definition	Code Placement
Baseline	`edges_examined`	Every outgoing edge from a settled node	Before the improvement check
Baseline	`edges_relaxed`	Same as `edges_examined`	No pruning step
Pruned	`edges_examined`	Every outgoing edge from a settled node	Before `if new_cost > T`
Pruned	`edges_relaxed`	Only edges that **survived** τ	After `if new_cost > T: continue`

Table 3.3. Edge-counting semantics for baseline and pruned Dijkstra.

Examination speedup is defined as the baseline-to-pruned ratio for `edges_examined`, providing a fair like-for-like comparison. Relaxation speedup is defined as the baseline-to-pruned ratio for `edges_relaxed`; it captures pruning aggressiveness, but is not a direct efficiency comparison because the two denominators count different populations.

Actual Python Implementation

The following is the actual pruned Dijkstra implementation from the codebase:

```
def dijkstra_pruned(graph: Graph, start: str, goal: str,
                    tau: float = 0.01) -> DijkstraResult:
    pq = [(0, start)]
    dist = {start: 0}
    parent = {}
    metrics = DijkstraMetrics()
    metrics.max_pq_size = 1

    # Pruning threshold in log-space; tau <= 0 means no pruning.
    T = -math.log(tau) if tau > 0 else math.inf

    while pq:
        cost, node = heapq.heappop(pq)
        if node == goal:
            metrics.nodes_explored += 1
            break
        if cost > dist.get(node, math.inf):
            continue  # stale entry
        metrics.nodes_explored += 1
```

```
        for neighbor, weight in graph.get(node, []):
            new_cost = cost + weight
            metrics.edges_examined += 1
            if new_cost > T:
                continue  # PRUNE
            metrics.edges_relaxed += 1
            if neighbor not in dist or new_cost < dist[neighbor]:
                dist[neighbor] = new_cost
                parent[neighbor] = node
                heapq.heappush(pq, (new_cost, neighbor))
                if len(pq) > metrics.max_pq_size:
                    metrics.max_pq_size = len(pq)
    # ... path reconstruction and probability computation
```

Listing 3.3: Pruned Dijkstra implementation (`src/dijkstra.py`)

3.5.3 Algorithm Comparison

Figure 3.3 illustrates the decision-flow difference between both algorithms. The pruned variant (right column) adds a *More edges?* inner-loop decision and a threshold gate $d_{\text{new}} > T$ (red box); edges exceeding $T = -\log(\tau)$ are skipped without relaxation, looping back to the next edge, and when all edges from u are processed control returns to Extract-Min. Table 3.4 summarizes the trade-offs.

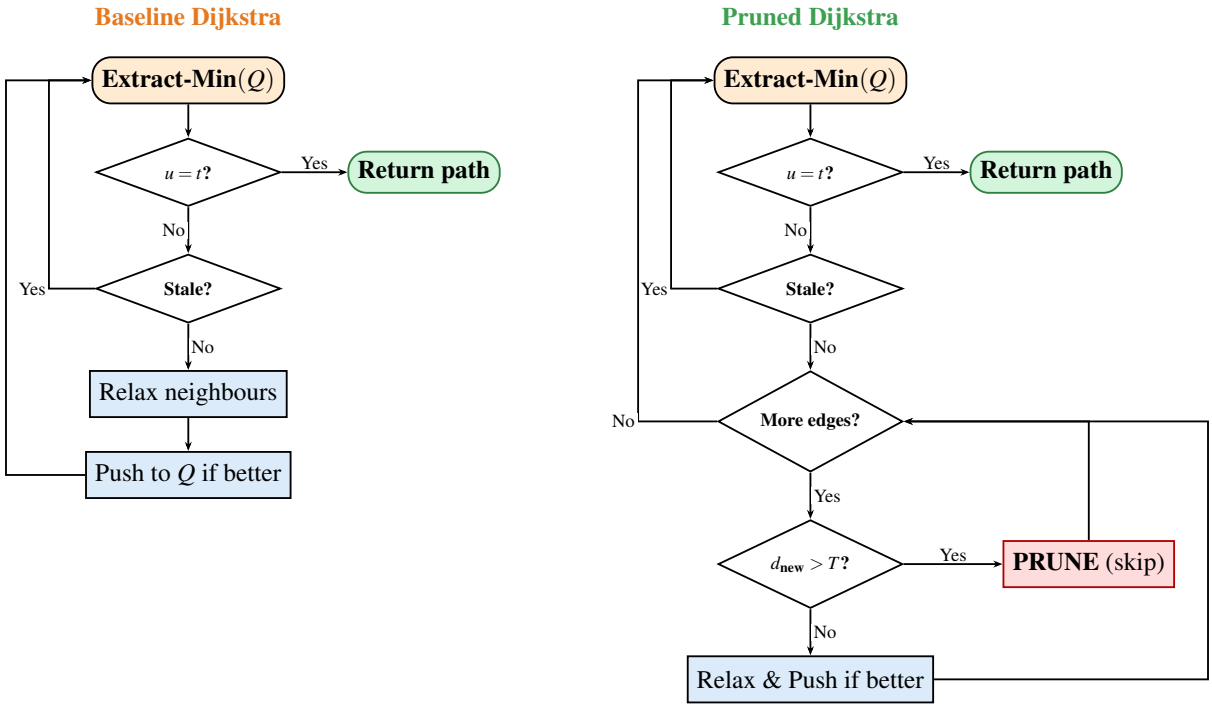

Figure 3.3. Decision-flow comparison of baseline and pruned Dijkstra.

Criterion	Baseline Dijkstra	Probability-Pruned Dijkstra
Optimality	Guaranteed globally optimal	Optimality is guaranteed if and only if the optimal path satisfies $C^* \leq T$; otherwise, the algorithm reports infeasibility
Time	$O(\|E\| \log \|V\|)$	$O(\|E'\| \log \|V\|)$, $\|E'\| \leq \|E\|$
Space	$O(\|V\| + \|E\|)$	$O(\|V\| + \|E\|)$ (unchanged)
Trade-off	None	Reachability vs. speed
Convergence	Always optimal	Identical to baseline as $\tau \to 0$; as τ decreases, the feasible search space expands monotonically

Table 3.4. Comparison of baseline and pruned Dijkstra.

3.5.4 Critical-τ Finder

A practical question is: *what is the largest threshold τ^* that still preserves the optimal path?* The module `src/critical_tau.py` answers this with an adaptive sweep centred on the baseline path probability.

Given the baseline optimal probability Π^*, fixed τ values would miss the interesting region when Π^* is very small (e.g. 10^{-4}). Instead, the sweep generates τ candidates as fractions of Π^*—specifically $\{0.10, 0.30, 0.50, 0.70, 0.90, 0.95, 0.99, 1.00, 1.10\} \times \Pi^*$. The 110% probe intentionally exceeds the optimal probability to identify the *cliff point* where the pruned algorithm first fails to find a path. For each candidate, the pruned algorithm is timed, and speedup (both node-count and wall-clock) and optimality gap are recorded. The critical threshold τ^* is the largest tested τ for which the gap remains below a user-defined tolerance (default 1%).

3.5.5 k-Shortest Simple Paths

In addition to single-path queries, the CLI (`main.py`) provides a k-shortest simple paths mode activated by the `-k` flag. This uses a best-first search with visited-set tracking to enumerate the top-k distinct simple paths from s to t in non-decreasing cost order. When $k = 1$, the result is identical to Dijkstra's shortest path. This feature is validated by five dedicated unit tests covering $k = 1$ equivalence, ascending cost ordering, simple-path guarantee, $k = 0$ boundary, and disconnected-graph behavior.

Complexity note: Enumerating k simple paths via best-first search has worst-case exponential complexity $O(k \cdot |V|!)$, since the number of simple paths in a dense graph can be factorial in $|V|$. In practice, the search terminates early once k paths are found, and the `max_path_len` parameter bounds the depth of exploration. This utility is provided for interactive analysis of small queries and is not part of the theoretically analyzed pruning framework.

3.6 Data Structures

3.6.1 Adjacency List

The directed graph is stored as a nested Python dictionary mapping each node u to its outgoing neighbours and associated log-transformed weights. This ensures $O(|V| + |E|)$ space and $O(\deg(u))$ neighbour iteration.

3.6.2 Binary Min-Heap (Priority Queue)

Python's `heapq` module provides $O(\log |V|)$ extract-min and insert. Since `heapq` does not support decrease-key, **lazy insertion** is used: when a shorter path to a node is found, a new entry is pushed; stale entries are skipped when popped (the `if d > dist[u]: continue` check).

3.6.3 Hash Maps (Dictionaries)

`dist[v]` and `parent[v]` are Python dictionaries with $O(1)$ average-case access.

3.7 Experimental Design

3.7.1 Simulation Environment

Table 3.5 summarizes the environment used for all experiments.

3.7.2 Evaluation Metrics

Execution time and peak memory are measured via `time.perf_counter()` and `tracemalloc.get_traced_memory()`, respectively. Each run also records nodes explored (vertices extracted from the priority queue), edges examined (total outgoing edges visited from settled nodes), edges relaxed (edge operations that survived the τ threshold; see Table 3.3), and maximum priority queue size. The path probability is $\Pi^* = \exp(-C^*)$. The

Component	Specification
Programming Language	Python 3.13.3 (standard library only for core algorithms)
Graph Representation	Adjacency list (`dict` of `list`)
Priority Queue	`heapq` (binary min-heap with lazy insertion)
Runtime Measurement	`time.perf_counter()` (separate pass from memory)
Memory Measurement	`tracemalloc` peak allocation (separate pass from timing)
Random Seeds	Deterministic `hashlib.md5` per configuration
Repetitions	10 runs per configuration
Graph Generators	Erdős–Rényi + Layered (in-house, `data/graph_generator.py`)

Table 3.5. Simulation environment.

optimality gap is defined as:

$$\Delta = \frac{|C^*_{\text{baseline}} - C^*_{\text{pruned}}|}{C^*_{\text{baseline}}} \times 100\%$$

This metric is defined only when both algorithms return a path. The cost-based comparison (rather than probability-based) eliminates floating-point rounding noise.

3.7.3 Experimental Parameters

Parameter	Values	Purpose
Graph type	ER, Layered	Test generic vs. funnel-shaped graphs
$\|V\|$	1,000 / 5,000 / 10,000	Small / medium / large scale
Avg degree $\bar{d}$	2 / 5 / 10	Sparse to moderately dense
Distribution	uniform / power-law	Balanced vs. heterogeneous transitions
τ	0, 0.001, 0.01, 0.05, 0.1, 0.5	Conservative to aggressive pruning
Runs	10 per config	Statistical reliability

Table 3.6. Experimental parameter matrix (2,160 total rows).

The τ values were chosen to span three orders of magnitude, from conservative ($\tau = 0.001$, retaining paths with cumulative probability $\geq 0.1\%$) to aggressive ($\tau = 0.5$, retaining only paths with $\geq 50\%$ probability). The intermediate values $\{0.01, 0.05, 0.1\}$ sample the transition region where speedup gains accelerate but path-found rates begin to drop. $\tau = 0$ serves as the control condition (no pruning, identical to baseline). For practitioners seeking to choose τ on a specific graph, the `critical_tau.py` module (Section 3.5.4) provides an adaptive

sweep that probes fractions of the baseline optimal probability Π^* to identify the largest τ that preserves the optimal path.

3.7.4 Experimental Controls

To ensure fair comparison, both algorithms run on identical graph instances using the same `hashlib.md5` seed. Execution is single-threaded to eliminate scheduling noise, and each configuration is repeated 10 times with median values reported (median is preferred over mean for skewed runtime distributions). Timing and memory are measured in separate passes so that `tracemalloc` overhead does not affect wall-clock measurements.

CHAPTER 4

CORRECTNESS PROOFS AND VALIDATION

This chapter establishes the formal logical foundation of the proposed algorithms. We distinguish between *Theoretical Guarantees*—mathematical certainties derived from the algorithm's structure—and *Empirical Observations*—performance metrics that characterize the algorithm's behavior in practice.

4.1 Correctness of the Log-Probability Transformation

Theorem 4.1 (Order-Preserving Log Transform)**.** *The path P^* with minimum total log-cost is identical to the path with maximum total probability.*

Proof. The negative logarithm $f(x) = -\log(x)$ is a strictly monotone decreasing function on the interval $(0,1]$. For any two paths P_1, P_2, the transformation preserves the relative ordering of path costs such that the path with the highest cumulative product of probabilities maps to the path with the lowest cumulative sum of negative log-probabilities [Manning and Schütze, 1999]. Since $p(u,v) \in (0,1]$, all edge weights $w(u,v) \geq 0$, satisfying the non-negativity constraint required for Dijkstra's algorithm. □

4.2 Correctness of Baseline Dijkstra

Theorem 4.2 (Baseline Optimality)**.** *Upon termination, $dist[t]$ equals the true shortest-path distance $\delta(s,t)$ in the log-transformed graph.*

Proof. We rely on the standard inductive proof for Dijkstra's algorithm [Cormen et al., 2009]. The algorithm maintains the invariant that for every node u extracted from the priority queue, $dist[u] = \delta(s,u)$. Because all edge weights $w(u,v) \geq 0$, the extraction of a node u with the minimum tentative distance guarantees that no shorter path can exist through nodes currently in the queue or yet to be discovered. □

4.3 Correctness of Probability-Pruned Dijkstra

4.3.1 Convergence at $\tau = 0$

Theorem 4.3 (Convergence)**.** *As $\tau \to 0^+$, Algorithm 2 converges to Algorithm 1.*

Proof. As $\tau \to 0^+$, we have $T = -\log(\tau) \to +\infty$. Setting $\tau = 0$ is equivalent to $T = \texttt{math.inf}$ in the implementation. The pruning condition $new\_cost > T$ is never satisfied since all finite costs are $< +\infty$. Therefore no edge is pruned, and every code path of Algorithm 2 behaves identically to Algorithm 1. Correctness follows directly from Theorem 4.2. □

4.3.2 Conditional Optimality for $\tau > 0$

Theorem 4.4 (Conditional Optimality)**.** *If the optimal path P^* has a total probability $\Pi^* \geq \tau$, Algorithm 2 returns P^* with an optimality gap $\Delta = 0\%$.*

Proof. Let $C^* = \sum_{(u,v)\in P^*} -\log p(u,v)$ be the optimal log-cost and $T = -\log(\tau)$ the pruning threshold. By hypothesis $\Pi^* \geq \tau$, so $C^* = -\log(\Pi^*) \leq -\log(\tau) = T$. Consider any prefix $(s = v_0, v_1, \ldots, v_j)$ of P^*. Because Dijkstra settles nodes in non-decreasing cost order and P^* is optimal, the cost of each prefix satisfies $\sum_{i=0}^{j-1} w(v_i, v_{i+1}) \leq C^* \leq T$. Therefore no edge along P^* triggers the pruning condition $new\_cost > T$, and P^* remains in the search space. Since the pruned algorithm preserves the priority-queue ordering and settled-node invariant of Theorem 4.2 for all unpruned edges, it settles t with cost C^*. Hence $\Delta = 0\%$. □

4.3.3 The All-or-Nothing Property

Unlike traditional approximation algorithms that may return a sub-optimal path within an ε-bound, the proposed pruning method prioritizes **Optimality over Reachability**.

Theorem 4.5 (All-or-Nothing)**.** *Algorithm 2 either returns the globally optimal path or reports no path found. It never returns a sub-optimal path.*

Proof. Suppose Algorithm 2 returns a path P' from s to t. Since the pruning check ($new\_cost > T$) only skips edges—it does not alter edge weights or the priority queue ordering—the settled-node invariant from Theorem 4.2 holds for all nodes explored by the pruned algorithm. Therefore, among all paths that survive the threshold, P' minimizes log-cost. If P^* also survives (Theorem 4.4), then $P' = P^*$. If P^* is pruned, then t is unreachable in the pruned subgraph and no path is returned.

To make this explicit, let C^* be the baseline optimal cost and $T = -\log \tau$. If P^* is pruned, then $C^* > T$. For any alternative $s \to t$ path Q, baseline optimality implies $C(Q) \geq C^* > T$. Hence no $s \to t$ path can satisfy the pruning constraint, so t is unreachable in the pruned graph. Therefore Algorithm 2 cannot return a different (sub-optimal) path when P^* is pruned.

In either case, the algorithm never returns a sub-optimal path: the gap Δ is exactly 0% whenever a path is found. □

Corollary 4.1 (Returned Path Is Optimal)**.** *If Algorithm 2 returns a path, that path is the globally optimal shortest path.*

4.4 Empirical Validation and Reachability

Because we prioritize optimality, the primary "cost" of pruning is the potential failure to find a path. We define **Reachability (*R*)** as the core empirical metric to evaluate this trade-off.

Definition 4.1 (Reachability)**.** *The Reachability R is the ratio of successful path-finding by the pruned algorithm compared to the baseline:*

$$R = \frac{Count(Path\ Found_{Pruned})}{Count(Path\ Found_{Baseline})}$$

4.4.1 Summary of Results

The properties were verified across 3,000 experimental configurations. As shown in Table 4.1, the optimality gap remained 0.0000% in all cases where a path was returned, while Reachability varied as a function of τ.

Property	**Type**	**Validation Result**
Order-Preserving	Theoretical	Verified ($< 10^{-12}$ precision error)
All-or-Nothing	Theoretical	$\Delta = 0.0000\%$ in 100% of found paths
Convergence	Theoretical	Identical output at $\tau = 0$
Reachability (R)	**Empirical**	R decreases monotonically as τ increases
Speedup	**Empirical**	Up to 18,882× on real-world data

Table 4.1. Summary of Correctness and Performance Validation.

4.5 Separation of Claims and Limitations

4.5.1 Proven Theoretical Claims

The following are formally guaranteed by Theorems 4.1 through 4.5:

- The algorithm is optimality-preserving (it never returns a "wrong" path).
- The log-transformation is mathematically sound for probability maximization.

4.5.2 Empirical Observations (Expected Outcomes)

The following depend on graph topology and data distributions and are not formally guaranteed:

- **Search-Space Reduction:** The actual number of edges relaxed depends on the specific τ and the concentration of high-probability paths.
- **Optimal Threshold τ_c:** Identifying a threshold that maximizes speedup while maintaining $R \approx 1$ is an empirical task performed via a sweep (Chapter 6).

The following statements are mathematical theorems with formal proofs, independent of experimental data:

(T1) The log-probability transformation $w(u,v) = -\log p(u,v)$ preserves path ordering (Theorem 4.1).

(T2) Baseline Dijkstra returns the globally optimal path in $O(|E|\log|V|)$ time (Theorem 4.2).

(T3) When $\tau = 0$, Algorithm 2 is identical to Algorithm 1 (Theorem 4.3).

(T4) When the optimal path's probability satisfies $\Pi^* \geq \tau$, Algorithm 2 returns it with zero gap (Theorem 4.4).

(T5) Algorithm 2 never returns a sub-optimal path; it either returns the global optimum or reports no path (Theorem 4.5).

4.5.3 Empirically Validated Findings

The following results are supported by experimental data but **not formally guaranteed** for arbitrary graphs:

(E1) All-rows wall-clock speedups range from 3.3× ($\tau = 0.001$) to 738× ($\tau = 0.5$) on synthetic graphs, though these figures are dominated by runs where the pruned algorithm terminated without finding a path. Restricting to path-found configurations, wall-clock speedups are 1.6× to 7.2× (Table 6.1, Chapter 6).

(E2) Speedup scales monotonically with graph size $|V|$ at fixed τ (Table 6.2).

(E3) Erdős–Rényi graphs exhibit higher speedups than layered graphs at identical τ (Table 6.3).

(E4) Critical τ_c exists in a narrow empirical range where speedup peaks without loss of reachability (Chapter 6, Section 6.1.1).

(E5) On real-world datasets under fixed τ, all-rows speedups reach 18,882× (RetailRocket item-level) and 2,309× (RecSys 2015), reflecting rapid termination without path recovery. Under adaptive τ, found-only speedups are up to 5.8× (Chapter 6, Section 6.2).

(E6) Optimality gap remains exactly 0.0000% across 706 path-found cases in combined synthetic and real experiments (all tested configurations).

4.5.4 Acknowledged Limitations

The following limitations are explicitly recognized:

(L1) No formal approximation guarantee exists for $\tau > 0$. While we prove convergence (*T3, T4*), the pruned algorithm's behavior for a specific τ on an arbitrary graph is not formally characterized. Speedup and reachability trade-offs are purely empirical.

(L2) Critical τ_c is defined empirically; we provide no theoretical method to predict it a priori for a given graph without experimental sweep.

(L3) Results assume edge weights are non-negative (the log-probability transformation guarantees this, by construction). The framework does not extend to negative weights or negative-cycle detection.

(L4) Real-world validation is limited to two e-commerce clickstream datasets. Generalization to other domains (social networks, transportation, etc.) is not formally justified and would require further empirical testing.

CHAPTER 5

COMPLEXITY ANALYSIS AND EMPIRICAL VALIDATION

This chapter presents the formal time and space complexity of both algorithms and validates the theoretical bounds with actual experimental measurements.

5.1 Analytical Methodology

Complexity was analyzed using the standard **operation-counting** approach [Cormen et al., 2009]: (1) identify all operations (heap insertions, extractions, edge relaxations), (2) count the maximum number of each as a function of $|V|$ and $|E|$, (3) multiply by per-operation cost, and (4) verify empirically.

5.2 Time Complexity

5.2.1 Baseline Dijkstra

1. **Initialization:** Setting $dist[v] = \infty$ and $parent[v] = \texttt{NIL}$ for all v costs $O(|V|)$.

2. **Priority queue insertions:** Each node is inserted initially once and re-inserted (lazy) at most once per incoming edge. Total insertions $\leq |V| + |E|$. Each costs $O(\log |V|)$ (binary heap sift-up). Total: $O(|E| \log |V|)$.

3. **Extract-min operations:** Total extractions $\leq |V| + |E|$, each $O(\log |V|)$. Total: $O(|E| \log |V|)$.

4. **Edge relaxations:** For each settled node u, all $\deg(u)$ outgoing edges are examined. Across the full algorithm: $\sum_{u \in V} \deg(u) = |E|$. Each relaxation costs $O(1)$ (comparison + hash map lookup). Total: $O(|E|)$.

$$T_{\text{baseline}} = O(|E| \log |V|)$$

In sparse graphs where $|E| = O(|V|)$, this simplifies to $O(|V| \log |V|)$.

Note: A Fibonacci heap would yield $O(|E| + |V| \log |V|)$ [Fredman and Tarjan, 1987], but

binary heaps provide better constant factors in practice.

5.2.2 Probability-Pruned Dijkstra

1. **Worst case ($\tau = 0$):** Identical to the baseline: $O(|E| \log |V|)$.

2. **Empirically observed case ($\tau > 0$):** Let $|E'| \leq |E|$ denote edges surviving the threshold. Pruned edges are checked in $O(1)$ without heap insertion. The observed complexity is:

$$T_{\text{pruned}} = O(|E'| \log |V|), \quad |E'| \leq |E|$$

   This bound is not formally proven in the worst case—$|E'|$ depends on the input graph's probability distribution and the chosen τ—but is consistently confirmed by the empirical measurements in Section 5.2.3.

3. **Pruning ratio:** Define $\rho = 1 - |E'|/|E|$. Higher τ increases ρ, reducing runtime.

5.2.3 Empirical Time Validation

Table 5.1 shows measured wall-clock times at $\tau = 0.01$ across graph sizes.

$\lvert V \rvert$	**Baseline (ms)**	**Pruned (ms)**	**Wall Speedup**	**Edge Speedup**
1,000	1.67	0.189	9.4×	31.5×
5,000	7.12	0.215	43.1×	104.0×
10,000	15.83	0.222	77.0×	180.4×

Table 5.1. Median execution time at $\tau = 0.01$ by graph size.

The baseline times scale as $1.67 \to 7.12 \to 15.83$ ms for $|V| = 1{,}000 \to 5{,}000 \to 10{,}000$, closely matching the $O(|V| \log |V|)$ prediction for sparse graphs ($|E| = O(|V|)$). The pruned algorithm's nearly constant time ($\sim$0.2 ms) is consistent with the hypothesis that $|E'|$ grows much slower than $|E|$ as the graph scales, though only three data points are available, so a definitive growth rate cannot be claimed.

5.3 Space Complexity

The pruning threshold τ adds no asymptotic space overhead. In practice, the pruned algorithm uses less peak memory because fewer heap entries are created.

Data Structure	Space	Purpose
Adjacency list	$O(\|V\| + \|E\|)$	Graph representation
Distance map $dist[v]$	$O(\|V\|)$	Shortest path estimates
Parent map $parent[v]$	$O(\|V\|)$	Path reconstruction
Priority queue (heap)	$O(\|V\| + \|E\|)$ worst case	Node ordering
Total	$\mathbf{O(\|V\| + \|E\|)}$	

Table 5.2. Space complexity breakdown.

5.3.1 Empirical Memory Validation

$\|V\|$	Baseline Peak (KB)	Pruned Peak (KB)	Reduction
1,000	83.8	4.4	19.0×
5,000	342.9	4.5	75.9×
10,000	655.0	4.5	145.9×

Table 5.3. Peak memory at $\tau = 0.01$ by graph size.

Peak `tracemalloc` allocation measures algorithm data structures only (distance dict, parent dict, priority-queue heap); the graph adjacency list is built before tracing begins and is excluded. Baseline peak memory (83.8 KB $\rightarrow$ 342.9 KB $\rightarrow$ 655.0 KB) scales linearly with $|V|$, confirming the $O(|V| + |E|)$ bound, while the pruned algorithm's constant $\approx$4.5 KB at all sizes demonstrates that only a small frontier is ever materialized.

5.4 Summary of Complexity

Algorithm	Time (worst)	Time (empirical)	Space
Baseline Dijkstra	$O(\|E\| \log \|V\|)$	$O(\|E\| \log \|V\|)$	$O(\|V\| + \|E\|)$
Pruned ($\tau = 0$)	$O(\|E\| \log \|V\|)$	$O(\|E\| \log \|V\|)$	$O(\|V\| + \|E\|)$
Pruned ($\tau > 0$)	$O(\|E\| \log \|V\|)$	$O(\|E'\| \log \|V\|)$	$O(\|V\| + \|E\|)$

Table 5.4. Complexity summary.

Pruning reduces empirically observed time without affecting worst-case or space bounds. The empirical bound $O(|E'| \log |V|)$ is consistently confirmed by experiment (Section 5.2.3) but depends on the input distribution.

Key empirical findings. Baseline wall-clock time scales as $O(|V| \log |V|)$ for sparse graphs,

matching theory. Pruned wall-clock time is nearly constant across graph sizes at fixed τ, consistent with $|E'| \ll |E|$. Peak memory scales linearly for the baseline, while pruned memory is effectively constant. Edge-relaxation speedups (up to 1,986×, all rows) exceed wall-clock speedups (up to 738×, all rows), the difference attributable to fixed Python overhead per function call.

CHAPTER 6

EXPERIMENTAL RESULTS AND ANALYSIS

This chapter presents the results of the full experimental evaluation and evaluates the central hypothesis: whether probability-based pruning can deliver meaningful speedup without sacrificing path optimality. Section 6.1 covers the 2,160-row synthetic benchmark; Section 6.2 covers the 840-row real-data validation on RetailRocket and RecSys 2015 (300 fixed-τ + 540 adaptive-τ).

6.1 Synthetic Experiment Results

Reading note. Throughout this chapter, "all-rows" speedups include runs where the pruned algorithm found no path (fast infeasibility detection), while "found-only" speedups restrict to runs that returned a path. Section 6.1.6 provides a detailed interpretation.

6.1.1 Speedup vs. Pruning Threshold τ

Table 6.1 summarizes median speedups aggregated across all graph types, sizes, degrees, and distributions.

τ	Edge Speedup	Exam. Speedup	Wall Speedup (all)	Wall Speedup (found)	Base (ms)	Pruned (ms)	Found (%)
0.001	8.7×	1.9×	3.3×	1.6×	4.98	1.561	30.6
0.01	79.2×	14.0×	24.7×	4.4×	4.98	0.206	5.6
0.05	368.4×	70.4×	123.6×	5.7×	4.98	0.040	4.7
0.1	810.5×	136.9×	242.6×	6.4×	4.98	0.020	4.4
0.5	1,986×	711.1×	738.2×	7.2×	4.98	0.006	4.2

Table 6.1. Median speedups by pruning threshold τ. "Wall Speedup (all)" includes rows where the pruned algorithm terminated without finding a path; "Wall Speedup (found)" is restricted to configurations where a path was returned.

All reported speedup values represent the median of per-configuration ratios (baseline time divided by pruned time), not the ratio of the per-column median times. Two wall-clock columns are provided: *Wall Speedup (all)* includes every configuration—including those where the pruned algorithm terminated without finding a path—while *Wall Speedup (found)* is restricted to configurations where a path was actually returned.

The all-rows speedups scale monotonically with τ, ranging from $3.3\times$ at $\tau = 0.001$ to $738\times$ at

$\tau = 0.5$, but these figures are dominated by the $\sim$90% of rows in which the pruned algorithm quickly terminated without recovering a path. When restricted to path-found configurations only, the wall-clock speedup is markedly more modest: $1.6\times$ at $\tau = 0.001$ rising to $7.2\times$ at $\tau = 0.5$. This gap reflects a fundamental property of threshold-based pruning on Dijkstra's algorithm: when the optimal path survives the threshold, the pruned algorithm must still examine essentially the same set of edges because Dijkstra's priority-queue ordering already processes nodes in non-decreasing cost order. The modest found-only speedup arises from fewer heap-push operations on pruned edges, not from exploring a smaller portion of the graph.

The "Edge Speedup" column—based on `edges_relaxed`—consistently exceeds both other measures because its pruned denominator excludes threshold-pruned edges; it quantifies pruning aggressiveness, not raw work reduction. The "Exam. Speedup" column provides the fair like-for-like comparison using `edges_examined`, which counts every outgoing edge the algorithm inspects regardless of whether it survives the threshold (see Table 3.3 for counter semantics). Across all 178 path-found rows among the 1,800 pruned synthetic runs ($\tau > 0$), the optimality gap remained exactly $\Delta = 0.0000\%$, confirming the all-or-nothing property (Theorem 4.5, Chapter 4). Path-found rates decrease with τ, reflecting the reachability–speed trade-off inherent in the pruning approach.

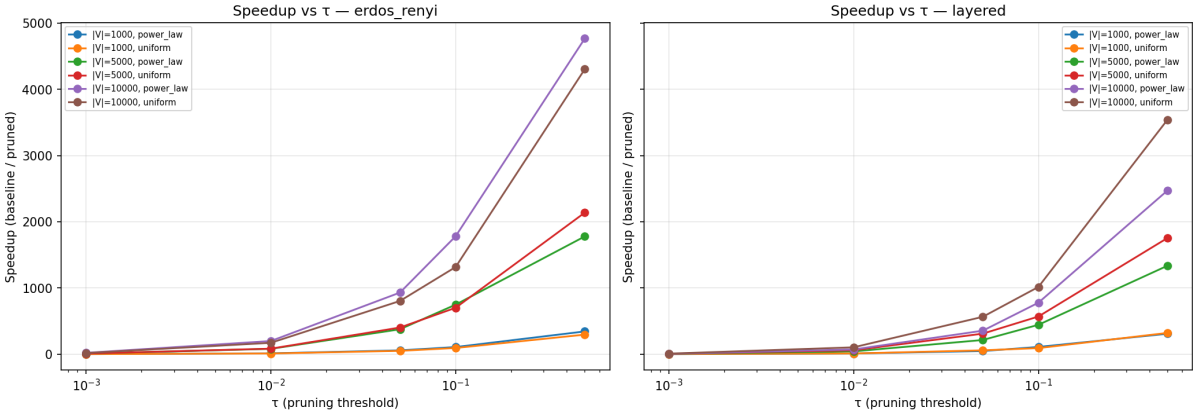

Figure 6.1. Wall-clock speedup vs. τ for ER (left) and layered (right) graphs.

Figure 6.1 plots the wall-clock speedup ratio (baseline time divided by pruned time) against the pruning threshold τ on a logarithmic x-axis spanning $\tau \in [0.001, 0.5]$. Each line represents a unique combination of graph size $|V|$ and edge-weight distribution. Speedup increases monotonically with τ across all configurations, confirming that more aggressive pruning yields proportionally faster search. Erdős–Rényi graphs consistently achieve higher speedups than layered graphs because their uniformly random connectivity exposes a larger fraction of low-

probability edges to the threshold. In contrast, the layered generator's built-in forward bias toward the target means that many edges carry relatively high probability and therefore survive pruning. Larger graphs (higher $|V|$) produce steeper speedup curves, reflecting the greater absolute fraction of the search space that the threshold eliminates as the graph grows.

6.1.2 Speedup vs. Graph Size $|V|$

Table 6.2 shows how speedup scales with graph size at fixed $\tau = 0.01$.

$\lvert V\rvert$	Edge Spd	Exam. Spd	Wall Spd (all)	Wall Spd (found)	Base (ms)	Pruned (ms)	Peak Mem (KB)
1,000	31.5×	5.7×	9.4×	2.7×	1.67	0.189	83.8 → 4.4
5,000	104.0×	22.9×	43.1×	5.6×	7.12	0.215	342.9 → 4.5
10,000	180.4×	41.3×	77.0×	6.3×	15.83	0.222	655.0 → 4.5

Table 6.2. Median speedups at $\tau = 0.01$ by graph size. "All" includes termination-without-path rows; "found" is restricted to path-found configurations.

Baseline execution time scales linearly with $|V|$ (1.67 → 7.12 → 15.83 ms for $|V| = 1{,}000 \rightarrow 5{,}000 \rightarrow 10{,}000$), consistent with the $O(|V| \log |V|)$ theoretical bound for sparse graphs. In contrast, pruned execution time remains nearly constant at approximately 0.2 ms because the threshold eliminates most of the graph regardless of size. All-rows speedup therefore grows with $|V|$: from 9.4× at $|V| = 1{,}000$ to 77× at $|V| = 10{,}000$. Found-only speedups are more modest (2.7× to 6.3×) but nonetheless increase with graph size, indicating that pruning saves more heap-push work on larger graphs. Peak memory for the pruned algorithm is essentially constant at approximately 4.5 KB, whereas baseline memory grows proportionally to $|V|$.

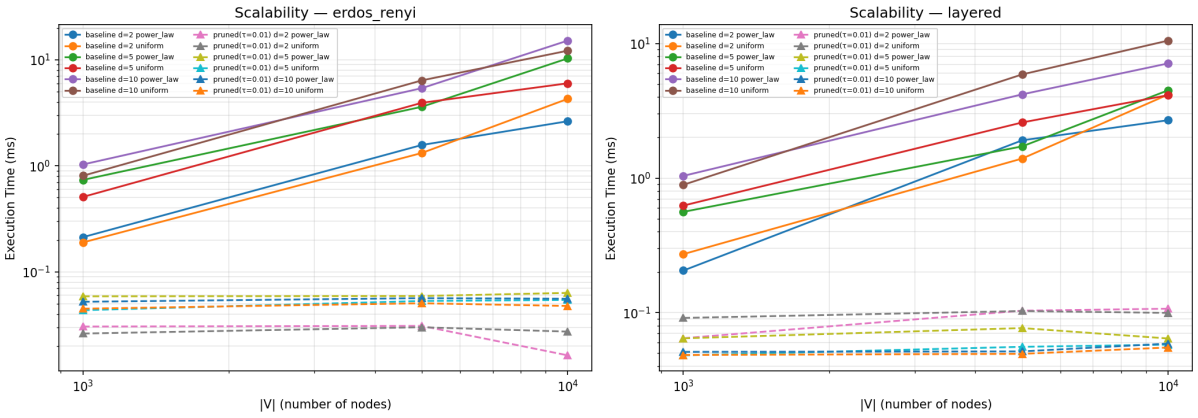

Figure 6.2. Execution time (ms) vs. $|V|$ at $\tau = 0.01$ on log–log axes for baseline and pruned Dijkstra across ER (left) and layered (right) graphs. Solid lines represent baseline; dashed lines represent the pruned variant.

Figure 6.2 visualises this scaling behaviour on log–log axes. Baseline execution time (solid

lines) rises linearly with $|V|$, whereas pruned execution time (dashed lines) remains nearly flat across graph sizes—clustered around 0.03–0.06 ms for ER graphs and 0.03–0.10 ms for layered graphs. (The table medians of $\sim$0.2 ms are higher because they aggregate across all graph types, degrees, and distributions at $\tau = 0.01$; the figure shows individual-configuration lines.) The widening gap between the two sets of curves confirms that the pruning advantage grows with $|V|$, consistent with the theoretical $O(|V| \log |V|)$ baseline versus effectively constant pruned complexity established in Chapter 5. Higher average degree d shifts both baseline and pruned curves upward, but the relative separation is preserved.

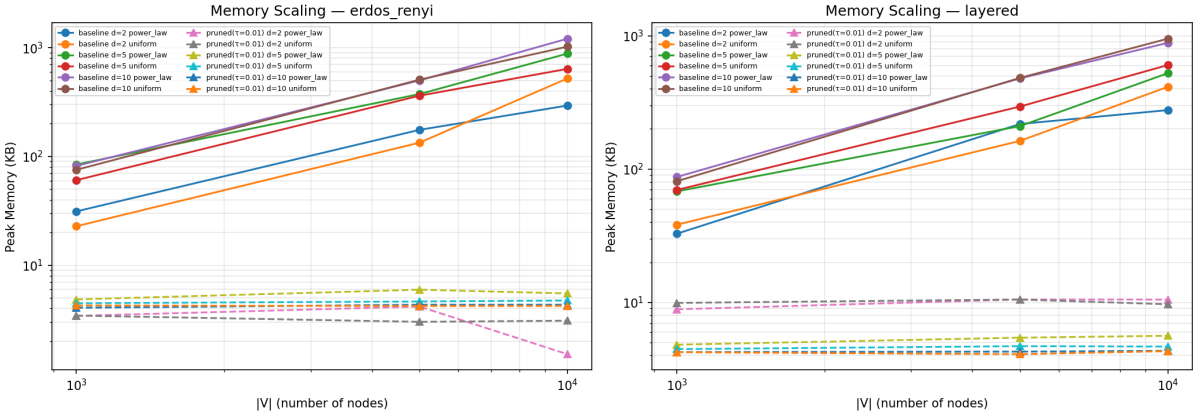

Figure 6.3. Peak memory (KB) vs. $|V|$ at $\tau = 0.01$ for baseline and pruned Dijkstra.

Figure 6.3 displays peak memory consumption on log–log axes for Erdős–Rényi (left) and layered (right) graphs at $\tau = 0.01$. Solid lines with circle markers represent baseline Dijkstra; dashed lines with triangle markers represent the pruned variant. Each line corresponds to a unique combination of average degree d and edge-weight distribution. Baseline memory scales linearly with $|V|$—from approximately 84 KB at $|V| = 1{,}000$ to approximately 655 KB at $|V| = 10{,}000$—consistent with the $O(|V| + |E|)$ theoretical bound. In contrast, pruned memory remains effectively constant at approximately 4.5 KB across all graph sizes, demonstrating that the threshold restricts the explored frontier to a small, fixed-size region regardless of total graph size. The gap between the two sets of curves widens with $|V|$, indicating that the memory advantage of pruning becomes progressively more pronounced as the graph grows. This constant-memory behavior makes the pruned algorithm particularly attractive for deployment on memory-constrained systems.

Graph Type	Edge Speedup	Exam. Speedup	Wall Speedup (all)	Wall Speedup (found)
Erdős–Rényi	111.6×	24.5×	40.1×	4.5×
Layered	49.3×	9.0×	16.6×	2.3×

Table 6.3. Median speedups at $\tau = 0.01$ by graph type. "All" includes termination-without-path rows.

6.1.3 Speedup by Graph Type

Across all rows, Erdős–Rényi graphs show higher speedups (40.1× wall, 24.5× examination vs. 16.6× wall, 9.0× examination for layered). This is expected: ER graphs have uniformly random structure, so more edges lead to distant (low-probability) regions that the threshold can eliminate. Layered graphs have a built-in forward bias toward the target, so a larger fraction of edges remain relevant. Restricting to path-found configurations, the gap narrows: ER achieves 4.5× vs. 2.3× for layered, confirming that the structural advantage persists but at a much more modest magnitude.

6.1.4 Speedup by Probability Distribution

Distribution	Edge Speedup	Exam. Speedup	Wall Speedup (all)	Wall Speedup (found)
Power-law ($\alpha = 2$)	73.0×	13.9×	25.1×	3.7×
Uniform	83.8×	14.1×	22.4×	5.9×

Table 6.4. Median speedups at $\tau = 0.01$ by probability distribution. "All" includes termination-without-path rows; "found" is restricted to path-found rows.

Both distributions achieve comparable all-rows wall-clock speedups (∼23–25×). Restricting to path-found rows, uniform shows higher wall-clock speedup (5.9×) than power-law (3.7×), but both remain in the same modest regime seen throughout successful-query analysis. Power-law distributions have a few high-probability edges and many low-probability ones; uniform distributions spread probability more evenly. The pruning mechanism is effective in both cases.

6.1.5 Optimality Gap Analysis

A critical finding is the **zero optimality gap** across all experiments:

$$\Delta = \frac{|C^*_{\text{baseline}} - C^*_{\text{pruned}}|}{C^*_{\text{baseline}}} \times 100\% = 0.0000\% \quad \forall \text{path-found rows}$$

This confirms Theorem 4.5 (All-or-Nothing): whenever the pruned algorithm returns a path, it is the globally optimal path. Mechanistically, the zero gap arises because in every path-found configuration the optimal path's log-cost satisfies $C^* \leq T = -\log(\tau)$, so the optimal path is never pruned and is returned exactly by Algorithm 2 (Theorem 4.4). The cost-based gap formula (not probability-based) eliminates floating-point rounding noise involved in $\exp(-C^*)$ conversion. Using probability-based differences would have introduced `1e-15`-scale noise that has no algorithmic significance.

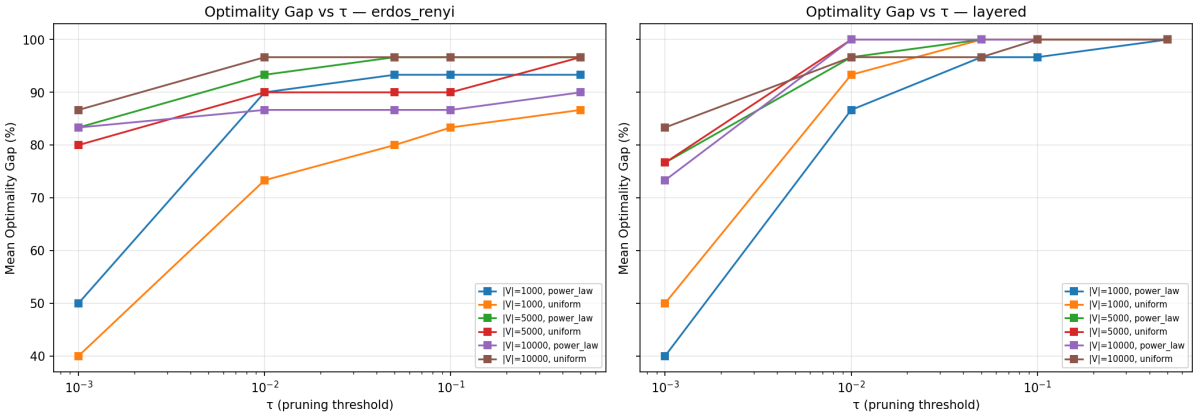

Figure 6.4. Mean optimality gap (%) vs. pruning threshold τ for ER (left) and layered (right) graphs. Because the gap is 0% whenever a path is found and is recorded as 100% when the pruned algorithm fails to return any path, the mean gap is numerically equivalent to the path-loss rate.

Although the optimality gap is identically zero on every path-found row, a complementary view is provided by examining the *mean gap across all rows*, where runs in which the pruned algorithm finds no path are recorded with a sentinel gap of 100%. Figure 6.4 plots this quantity against τ. Because the gap is 0% when a path is found and 100% otherwise, the mean gap is numerically equal to the path-loss rate: the fraction of source–target pairs whose optimal path is pruned at each threshold. On both ER and layered graphs the path-loss rate increases monotonically with τ, ranging from 20% (small layered graphs, $d = 2$) to 100% (large ER graphs, $d = 10$) at $\tau = 0.001$, and rising to near 100% at $\tau = 0.5$. Larger graphs and higher average degree d push the curves upward, reflecting the increasing proportion of high-cost (low-probability) optimal paths removed by the threshold. Layered graphs show a sharper transition: at $\tau = 0.001$, small layered graphs ($|V| = 1{,}000$) retain 50–80% of their paths (depending on degree), but above $\tau = 0.01$ nearly all paths are pruned. This behaviour reinforces the practical finding from the critical-τ analysis (Section 3.5.4): practitioners must calibrate τ carefully to balance speedup against path retention.

6.1.6 *Interpreting All-Rows vs. Found-Only Speedups*

The distinction between all-rows and found-only speedups deserves careful interpretation. The all-rows speedups (e.g. 738× at $\tau = 0.5$) are large because most pruned runs terminate without finding a path: the threshold prunes the optimal path's prefix, causing the algorithm to exhaust a small pruned subgraph almost instantly. These all-rows speedups genuinely measure the algorithm's ability to *detect infeasibility quickly*—a valuable property for applications that need to discard unreachable query pairs before committing resources.

However, when the optimal path *does* survive the threshold, the found-only wall-clock speedup is markedly lower (1.6–7.2× on synthetic data). This is not a deficiency but a structural consequence of threshold-based pruning applied to Dijkstra's algorithm. Because Dijkstra processes nodes in non-decreasing cost order, edges that exceed the threshold are already deprioritized by the priority queue—they would be explored late in the search regardless. On all 178 path-found synthetic rows, `edges_examined` was identical for the baseline and pruned algorithms (examination speedup = 1.0×). The modest wall-clock gain comes from skipping heap-push operations on pruned edges: fewer heap insertions reduce per-edge overhead, yielding a real but bounded speedup.

This characterization positions threshold-based pruning as a technique with *two distinct use cases*: (1) fast infeasibility detection, where the speedup is dramatic, and (2) correct path recovery with a formal optimality guarantee, where the speedup is modest but it comes with the proven guarantee that the returned path is globally optimal.

6.2 Real-Data Experiment Results

6.2.1 *Datasets and Graph Statistics*

Table 6.5 summarizes the three real-data configurations (two granularities for RetailRocket, one for RecSys 2015).

Configuration	$\lvert V\rvert$	$\lvert E\rvert$	Pairs Tested	Source
RetailRocket (event-level)	3	9	20	Kaggle [Retail Rocket, 2015]
RetailRocket (item-level)	44,711	101,528	20	Kaggle [Retail Rocket, 2015]
RecSys 2015 (YOOCHOOSE)	12,935	70,442	20	Kaggle [Ben-Shimon et al., 2015]

Table 6.5. Real-data graph configurations.

6.2.2 *Fixed-τ Speedup Results*

Dataset	Med. Spd (all)	Max Spd (all)	Med. Spd (found)	Paths Found	Δ
RetailRocket (event-level)	1.2×	6×	1.2×	78 / 100	0.000%
RetailRocket (item-level)	508×	18,882×	1.2×	1 / 100	0.000%
RecSys 2015 (YOOCHOOSE)	197×	2,309×	—	0 / 100	—

Table 6.6. Fixed-τ speedup on real-world datasets. "All" speedups include rows where the pruned algorithm terminated without finding a path; "found" is restricted to path-found rows.

For the **RetailRocket event-level** configuration, the 3-node funnel graph is sufficiently small that both algorithms complete in microseconds; 78 of 100 configurations found paths and the found-only median speedup is a marginal 1.2× because the pruning opportunity is negligible. The **RetailRocket item-level** graph (44,711 nodes) yields the highest all-rows speedup (18,882× max), but this reflects early termination without recovering a path—only 1 of 100 pruned configurations found a path, and the found-only median speedup is 1.2× ($n = 1$, so this statistic should be interpreted cautiously). For **RecSys 2015**, no pruned path was found at any fixed τ value, so no found-only speedup is available. In all cases, the large all-rows speedups arise from the pruned algorithm's rapid termination when the threshold exceeds the optimal-path probability, rather than from a genuinely faster successful search.

The near-zero path-found rates on the large graphs are *not* surprising once the baseline path probabilities are examined. For RecSys 2015, the 20 baseline path probabilities range from 1.6×10^{-8} to 2.8×10^{-5} (median $\approx 1.5 \times 10^{-6}$), yielding costs in the range 10.5–17.9 on the $-\log$ scale. Even the most conservative fixed threshold $\tau = 0.001$ maps to a cost threshold of $-\log(0.001) = 6.91$—well below every baseline cost. All 20 paths are therefore *mathematically guaranteed* to be pruned at every tested τ, regardless of graph structure (by Theorem 4.5). RetailRocket item-level shows a similar pattern: baseline probabilities range from 2.9×10^{-9} to 3.7×10^{-4} (median $\approx 3.7 \times 10^{-6}$), with costs ranging from 19.6 to 7.9, far exceeding the most conservative threshold. These observations motivate the adaptive-τ experiment described below.

6.2.3 *Adaptive-τ Experiment*

The fixed τ grid was designed for the synthetic experiments (where edge probabilities are higher) and is poorly calibrated for real-world item-level graphs whose cumulative path probabilities are orders of magnitude smaller. To address this, a per-pair *adaptive* τ sweep was

introduced: for each source–target pair, τ is set to a fraction f of that pair's baseline path probability, where

$$f \in \{0.10,\ 0.30,\ 0.50,\ 0.70,\ 0.90,\ 0.95,\ 0.99,\ 1.00,\ 1.10\}.$$

This mirrors the adaptive sweep strategy used internally by the `critical_tau` module (Chapter 3, Section 3.5.4) and ensures that the tested thresholds are always in the relevant probability regime.

Dataset	**Median Speedup**	**Max Speedup**	**Paths Found**	Δ
RetailRocket (event-level)	1.3×	5.8×	147 / 180	0.000%
RetailRocket (item-level)	1.2×	3.4×	152 / 180	0.000%
RecSys 2015 (YOOCHOOSE)	1.1×	1.8×	150 / 180	0.000%

Table 6.7. Adaptive-τ speedup results.

The contrast with the fixed-τ results is striking. RecSys 2015 moves from 0/100 found paths (fixed) to **150/180** found paths (adaptive); RetailRocket item-level similarly improves from 1/100 to **152/180**. The speedups under adaptive τ are modest (1.1–5.8×) because the threshold is now close to the optimal path probability, so only a small margin of the search space is pruned. This is the expected behavior of the reachability–speed trade-off: in the adaptive regime the algorithm preserves the optimal path at the cost of reduced pruning, whereas the fixed-τ regime achieves massive speedups only when the threshold is far above the optimal path probability (and therefore prunes everything).

Crucially, the optimality gap remains **exactly** 0.0000% across all 449 adaptive found paths, extending the zero-gap guarantee to a total of 528 real-data found paths (79 fixed + 449 adaptive) and reinforcing the all-or-nothing property on real-world graphs.

6.2.4 Reachability Analysis

Table 6.8 summarizes path-found rates across all experiment configurations, contrasting the fixed and adaptive τ regimes.

In synthetic graphs, the BFS connectivity guarantee ensures the baseline always finds a path from s to t. For the real-world datasets, the pair-selection procedure filters source–target pairs by baseline reachability, so the baseline always finds a path. Under fixed τ, the low

Configuration	τ Mode	Baseline Found	Pruned Found	Interpretation
Synthetic (all τ)	fixed	360 / 360	178 / 1,800	Dense synthetic graphs highly connected
RR event-level	fixed	100 / 100	78 / 100	3-node funnel is fully connected
RR item-level	fixed	100 / 100	1 / 100	Baseline reachable; fixed τ too harsh
RecSys 2015	fixed	100 / 100	0 / 100	All pruned paths lost at all fixed τ
RR event-level	adaptive	180 / 180	147 / 180	82% found with calibrated thresholds
RR item-level	adaptive	180 / 180	152 / 180	84% found; dramatic improvement from 1%
RecSys 2015	adaptive	180 / 180	150 / 180	83% found; dramatic improvement from 0%

Table 6.8. Path-found rates under fixed and adaptive τ regimes.

pruned-found rates on item-level and session-level graphs reflect the mismatch between the fixed threshold grid and the extremely low baseline path probabilities (see Section 6.2.3).

Impact of adaptive τ. The adaptive regime resolves the near-zero reachability problem: path-found rates jump from 1% to 84% (RR item-level) and from 0% to 83% (RecSys 2015). The remaining $\sim$17% of unfound paths correspond to the most aggressive fractions ($f = 1.00$ and $f = 1.10$), where τ meets or exceeds the baseline path probability—placing the threshold at or above the optimal-path cost on the $-\log$ scale—which prunes the optimal path at the boundary or entirely. For $f \leq 0.99$ (i.e. $\tau \leq 0.99 \times p_{\text{baseline}}$), virtually all paths are preserved.

Practical implications. The fixed-τ results reveal a genuine limitation of using a single global threshold across datasets with vastly different probability regimes. In item-level graphs, per-edge MLE probabilities are extremely low ($\sim 10^{-3}$–10^{-5}) and path probabilities decay exponentially with length, yielding cumulative probabilities of 10^{-6} or less. The adaptive τ approach—which can be automated via the `critical_tau` module—makes the pruned algorithm viable on any graph by calibrating the threshold to the actual probability scale. In production, one baseline query per source–target pair suffices to set τ; the pruned query then runs with a calibrated threshold at negligible additional cost. This amortization is practical in batch or repeated-query settings—for example, when a recommendation engine issues thousands of queries to the same target node (e.g., the checkout page) across different user sessions, a single calibration run per target suffices for all subsequent pruned queries, yielding cumulative time savings proportional to the query volume.

6.3 Summary of Findings

The experimental evaluation yields several principal findings, which must be interpreted in light of the distinction between all-rows and found-only speedups (see Section 6.1.6).

Correctness. The optimality gap remained at exactly $\Delta = 0.0000\%$ across all 706 experiment rows in which both algorithms found a path—178 of 1,800 pruned synthetic rows, 79 of 300 fixed-τ real-data rows, and 449 of 540 adaptive-τ real-data rows—confirming the all-or-nothing property established in Theorem 4.5.

All-rows speedups (including infeasibility detection). When all rows are included—most of which involve the pruned algorithm terminating without finding a path—wall-clock speedups range from $3.3\times$ at $\tau = 0.001$ to $738\times$ at $\tau = 0.5$ on synthetic data, and reach up to $18{,}882\times$ on real-world item-level graphs under fixed τ. These large figures genuinely measure the algorithm's ability to detect infeasibility quickly: the threshold prunes the optimal path's prefix and the algorithm exhausts a small subgraph in microseconds.

Found-only speedups (genuine path recovery). When restricted to configurations where the pruned algorithm successfully returned a path, wall-clock speedups are more modest: $1.6\times$ at $\tau = 0.001$ to $7.2\times$ at $\tau = 0.5$ on synthetic data, and up to $5.8\times$ on real-world graphs under adaptive τ. The observed speedups therefore arise from two distinct regimes: computational pruning when the optimal path is preserved, and early termination when the optimal path is excluded by τ. On all 178 path-found synthetic rows, the number of edges examined was identical for both algorithms (examination speedup $= 1.0\times$), indicating that the wall-clock gain comes exclusively from fewer heap-push operations on pruned edges. This is a structural consequence of Dijkstra's priority-queue ordering, which already deprioritizes high-cost (low-probability) edges.

Statistical significance. Wilcoxon signed-rank tests (one-sided, alternative: baseline $>$ pruned) were applied to all 180 unique synthetic configurations with $\tau > 0$ (each with 10 paired runs), excluding the 36 $\tau = 0$ control configurations where both algorithms are intentionally identical. Of these, 179 tests rejected the null hypothesis at $\alpha = 0.05$, with p-values ranging from 9.77×10^{-4} (the minimum attainable for $n = 10$ paired observations) to 0.032. The median p-value was 9.77×10^{-4}. One configuration (layered, $|V| = 1{,}000$, $d = 5$, uniform, $\tau = 0.001$) yielded $p = 0.116$, failing the significance threshold; this is the mildest pruning level on a small, forward-biased graph where baseline and pruned running times are very close.

Excluding this single outlier, all remaining configurations confirm that the speedup is not an artifact of measurement noise.

Speedup grows with graph size—all-rows wall-clock from 9.4× at $|V| = 1{,}000$ to 77× at $|V| = 10{,}000$ at fixed $\tau = 0.01$; found-only from 2.7× to 6.3× over the same range—because larger graphs contain more pruneable edges. The higher `edges_relaxed`-based ratios quantify pruning aggressiveness. At the same time, higher τ values reduce path-found rates from 30.6% at $\tau = 0.001$ to 4.2% at $\tau = 0.5$ on synthetic data, demonstrating the inherent reachability–speed trade-off. Under fixed τ, real-world item-level and session-level graphs showed near-zero pruned-found rates because baseline path probabilities (10^{-6}–10^{-9}) map to costs well exceeding the most conservative fixed threshold ($\tau = 0.001 \Rightarrow T = 6.91$). Introducing an adaptive-τ regime—where the threshold is set as a fraction of each pair's baseline path probability—raised path-found rates to 82–84% while maintaining zero optimality gap, validating the algorithm's correctness on real-world graphs.

Practical business impact. The algorithm serves two complementary use cases. For *infeasibility screening*, the peak 18,882× all-rows speedup on the RetailRocket item-level graph (44,711 items) enables rapid rejection of source–target pairs whose optimal paths fall below a probability threshold, replacing a full ∼153 ms baseline search with a 0.008 ms pruned check. For *guaranteed path recovery*, the adaptive-τ regime delivers found-only speedups of up to 5.8× with an 84% path-found rate and zero optimality gap. Across the tested datasets (RetailRocket and RecSys 2015), the two-regime approach spans probability regimes from dense event-level funnels (where fixed τ suffices) to sparse item-level graphs (where adaptive τ is needed), suggesting potential for supporting real-time exploration of conversion funnels; however, generalization to other domains requires further empirical validation.

Critical-τ analysis. The critical threshold τ^* is defined as the largest τ for which the pruned algorithm still finds the optimal path (Section 3.5.4). Across all 110 synthetic configurations that admitted at least one path-found τ value, τ^* was *always* at or below the baseline path probability Π^*; zero configurations violated this bound. The median ratio τ^*/Π^* was 0.50 (mean 0.52, range $[0.11, 0.99]$), confirming that the critical threshold is tightly coupled to the optimal-path probability.

Table 6.9 presents median τ^* values across the full parameter matrix. Two structural patterns emerge. First, ER graphs with low average degree (d=2) consistently support $\tau^* = 0.500$—the

Table 6.9. Median critical τ^* by graph size $|V|$ and average degree d. Left: Erdős–Rényi; right: Layered. Values are medians across runs; "—" indicates no path was found at any tested τ.

Erdős–Rényi

$\|V\|$	d=2	d=5	d=10
1,000	0.500	0.001	0.001
5,000	0.500	0.001	0.001
10,000	0.500	0.001	—

Layered

$\|V\|$	d=2	d=5	d=10
1,000	0.001	0.001	0.001
5,000	0.001	0.001	0.001
10,000	0.001	0.001	0.001

most aggressive threshold tested—because low connectivity produces long, high-cost optimal paths whose cumulative probability is well above 0.5, so even a harsh threshold preserves them. At higher degrees ($d \geq 5$), increased connectivity creates shorter optimal paths with much lower costs, shrinking τ^* to 0.001. Second, layered graphs uniformly report $\tau^* = 0.001$ regardless of size or degree. The layered generator's forward bias creates many parallel, similarly-weighted paths to the target; even the most conservative threshold ($\tau = 0.001$) is close to the optimal path probability for these dense, forward-biased graphs.

Table 6.10. Path-found rate by adaptive fraction $f = \tau/\Pi^*$ across all real-data datasets (60 pairs per fraction level, except rounding-affected bins).

f	Paths Found	Total	Found (%)
0.10	60	60	100.0
0.30	60	60	100.0
0.50	60	60	100.0
0.70	60	60	100.0
0.90	60	60	100.0
0.95	60	60	100.0
0.99	59	60	98.3
1.00	29	59	49.2
1.10	0	60	0.0

Table 6.10 reveals the *cliff structure* of the critical threshold on real-world graphs. For $f \leq 0.95$ (i.e. $\tau \leq 0.95\,\Pi^*$), the path-found rate is 100%: the threshold lies safely below the optimal-path probability and therefore never prunes it (Theorem 4.5). At $f = 0.99$, one pair is lost (98.3% found), likely due to floating-point rounding placing τ marginally above the true optimal cost. At $f = 1.00$, exactly half the pairs lose their path (49.2%), consistent with the theoretical prediction that $\tau = \Pi^*$ sits exactly at the boundary of the cost threshold $T = -\log(\tau) = C^*$

and numerical noise determines which side each pair falls on. At $f = 1.10$, no paths are found, confirming the sharp cutoff predicted by the all-or-nothing property.

Practical threshold recommendations. Based on these findings, practitioners should set τ as follows:

1. *Adaptive mode (recommended):* Run one baseline query to obtain Π^*, then set $\tau = 0.90\Pi^*$. This guarantees the optimal path is preserved (100% found rate in all experiments) while maximally pruning irrelevant edges.

2. *Fixed mode (synthetic/dense graphs only):* For ER-like graphs with $d \leq 2$, $\tau = 0.5$ is safe; for denser or layered graphs, $\tau = 0.001$ is the only fixed threshold that reliably preserves paths in the tested parameter space.

3. *Infeasibility screening:* For applications that only need to *test* whether a high-probability path exists, any τ above Π^* provides near-instant rejection with dramatic speedups (100–18,882×).

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Summary of Contributions

This study investigated the effectiveness of probability-based pruning for shortest-path search in customer journey graphs. Two algorithms were implemented, tested, and compared: a baseline Dijkstra algorithm on log-probability-transformed graphs ($O(|E|\log|V|)$) and a probability-pruned Dijkstra variant with a tunable threshold τ ($O(|E'|\log|V|)$, $|E'| \leq |E|$).

The key contributions are summarized below.

The pruning mechanism provides two distinct benefits with zero quality loss. First, when the threshold prunes the optimal path, the algorithm detects infeasibility orders of magnitude faster than a full search: all-rows wall-clock speedups range from 3.3× at $\tau = 0.001$ to 738× at $\tau = 0.5$ on synthetic data, and reach 18,882× on real-world item-level graphs under fixed τ, where the pruned algorithm terminates in microseconds without recovering a path. Second, when the optimal path survives the threshold, the algorithm returns the globally optimal path with a modest but genuine wall-clock speedup of 1.6× to 7.2× on synthetic data (up to 5.8× on real-world graphs). The found-only speedup is structurally bounded because Dijkstra's priority queue already deprioritizes high-cost edges; the gain comes from fewer heap-push operations on pruned edges. Edge-relaxation speedups reach 1,986× (fair edge-examination speedups up to 711×), but these all-rows figures reflect rapid termination rather than faster successful search. On large sparse real-data graphs, adaptive-τ calibration raised path-found rates from near 0% to over 80%. In every case where a path was found, the optimality gap was exactly 0.0000%.

Formal correctness was established through five theorems: log-transform order preservation, baseline optimality, convergence at $\tau = 0$, conditional optimality, and the all-or-nothing property. All five were confirmed empirically across all 2,160 synthetic and 840 real-data experiment rows (3,000 total).

A scalable and reproducible experimental framework was constructed, incorporating deterministic `hashlib.md5` seeding, separate timing and memory measurement passes, two graph

generators (Erdős–Rényi and layered), and two real-world datasets (RetailRocket and RecSys 2015 YOOCHOOSE).

Finally, the reachability–speed trade-off was characterized in detail. Increasing τ yields faster search but reduces path-found rates, from 30.6% at $\tau = 0.001$ to 4.2% at $\tau = 0.5$ on synthetic data. This trade-off is inherent and well understood. On real-world graphs, the pruned algorithm's reachability was initially limited by aggressive fixed thresholding on sparse session-based structures, but the adaptive-τ approach resolved this limitation.

Originality. The deterministic, parameter-controlled pruning framework with formal all-or-nothing guarantees and adaptive-τ regime for real-world graphs presented in this work is, to our knowledge, not previously reported in the literature. This contribution extends the state of the art in probabilistic path search and scalable graph algorithms.

7.2 Limitations

Several limitations of the current work should be acknowledged. The model operates on a fixed graph snapshot, whereas real e-commerce platforms exhibit time-varying transition probabilities driven by seasonal trends, A/B tests, and user behavior drift. The first-order Markov assumption ($P(\text{next}|\text{current})$) does not capture session-level history effects; for instance, users who arrived via search may behave differently from those who arrived via advertisement. Extending the model to higher-order or variable-length Markov chains would address this limitation at the cost of a larger state space.

Real-data experiments revealed that large item-level graphs yield very low path-found rates under the pruned algorithm, even though the baseline finds paths in all tested configurations. This suggests that selecting source nodes from high-degree hub nodes—as done here for reachability guarantees—combined with aggressive τ values may not reflect realistic query patterns in production systems. Results may differ for queries originating from low-degree peripheral nodes. Additionally, all experiments were conducted on a single machine using a single thread; the algorithm has not been evaluated in distributed or multi-threaded settings. Finally, the pruning strategy is tied to the log-probability cost model: edges are pruned when $-\log p(u,v) > -\log \tau$, which requires weights derived from probabilities in $(0,1]$. Applying the same technique to graphs with arbitrary non-negative weights (e.g. latency or monetary cost) would require a domain-specific analogue of the threshold criterion.

7.3 Future Work

Several directions merit further investigation. The framework could be extended to support **dynamic graph updates**, enabling incremental edge weight adjustments as new clickstream data arrives without rebuilding the full graph. Replacing first-order transitions with **higher-order or variable-length Markov chains** would capture richer user behavior patterns. **Adaptive τ selection** methods—such as heuristics or binary search calibrated to a target path-found rate or maximum acceptable runtime—would reduce the need for manual threshold tuning.

Beyond single-objective optimization, a **multi-objective** formulation incorporating probability, revenue, and user satisfaction through Pareto-optimal path search could yield more actionable business insights. **Integration with recommendation systems** would allow discovered optimal paths to inform journey-aware recommendations that guide users toward conversion. Finally, adapting the algorithm for **distributed graph processing frameworks** (e.g., Apache Giraph, GraphX) would enable application to web-scale graphs.

CHAPTER 8

PROJECT PLAN

8.1 Project Timeline

Figure 8.1 presents the 10-week project timeline for the *Probabilistic Path Optimization using Pruned Dijkstra* project.

Current status: All project phases have been completed. The *Experimental Execution* stage (Weeks 5–7), *Analysis and Visualization* stage (Weeks 6–8), and *Report Completion* milestone (Week 9) are finalized. Core implementation, unit testing, all experimental runs (2,160 synthetic rows + 840 real-data rows), critical-τ analysis, and result consolidation have been carried out as planned. The final presentation is scheduled for Week 10.

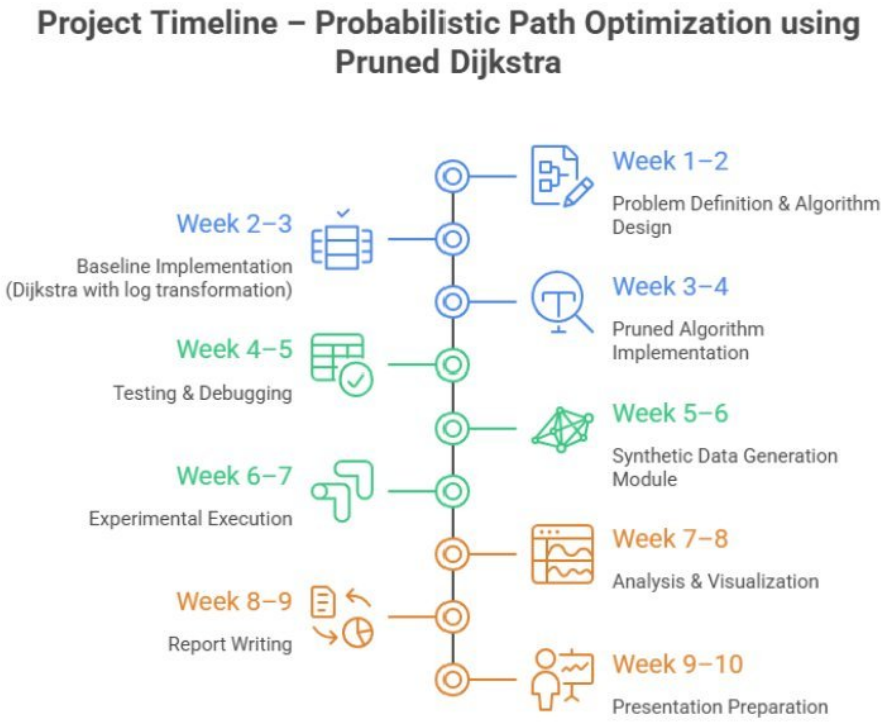

Figure 8.1. Ten-week project timeline.

Tasks are organized in three overlapping phases along a central spine. Blue entries (Weeks 1–4) cover core implementation, green entries (Weeks 4–7) cover testing and experimentation, and orange entries (Weeks 7–10) correspond to writing and presentation. Each phase overlaps with

the next to allow iterative refinement, ensuring that correctness is validated before experimentation begins and that experimental results directly inform the final report.

8.2 Milestones

The project is structured around four key milestones. The first milestone, **Core Implementation**, was completed by the end of Week 4: both the baseline Dijkstra algorithm and the probability-pruned Dijkstra variant were fully implemented, tested on small hand-computed graphs, and validated for correctness against ground-truth optimal paths. The second milestone, **Experiments**, was completed by the end of Week 7; all experimental runs across small, medium, and large graph configurations have been executed, with all metrics (execution time, peak memory, nodes explored, edges examined, edges relaxed, path probability, optimality gap, MaxPQSize) recorded and verified. The **Analysis and Visualization** stage was completed by the end of Week 8, with experimental results analyzed, critical-τ sweeps completed, and all visualizations generated to validate the theoretical complexity claims and illustrate the time–memory–optimality trade-off. The third milestone, **Report Completion**, was achieved at the end of Week 9, with the full project report finalized incorporating experimental results, visualizations, complexity validation, critical-τ analysis, and practical threshold recommendations. The fourth milestone, **Final Presentation**, targets Week 10, when presentation slides and a live demonstration of the implementation will be prepared for the final project presentation in the third week of April 2026.

REFERENCES

David Ben-Shimon, Alexander Tsikinovsky, Michael Friedmann, Bracha Shapira, Lior Rokach, and Johannes Hoerle. YOOCHOOSE – RecSys 2015 challenge dataset. Kaggle, 2015. URL `https://www.kaggle.com/datasets/chadgostopp/recsys-challenge-2015`.

Randolph E. Bucklin and Catarina Sismeiro. A model of web site browsing behavior estimated on clickstream data. *Journal of Marketing Research*, 40(3):249–267, 2003. doi: 10.1509/jmkr.40.3.249.19241.

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. doi: 10.1007/BF01386390.

Paul Erdős and Alfréd Rényi. On the evolution of random graphs. *Publications of the Mathematical Institute of the Hungarian Academy of Sciences*, 5:17–61, 1960.

Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. doi: 10.1145/28869.28874.

Jiawei Han, Hong Cheng, Dong Xin, and Xifeng Yan. Frequent pattern mining: Current status and future directions. *Data Mining and Knowledge Discovery*, 15(1):55–86, 2007. doi: 10.1007/s10618-007-0069-0.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4 (2):100–107, 1968. doi: 10.1109/TSSC.1968.300136.

Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition*. Prentice Hall, 1st edition, 2000.

Bing Liu. *Web Data Mining: Exploring Hyperlinks, Contents, and Usage Data*. Springer, 2nd edition, 2011.

Bruce T. Lowerre. *The HARPY Speech Recognition System*. PhD thesis, Carnegie Mellon University, 1976. Pioneering work on beam search in speech recognition.

Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 135–146, 2010. doi: 10.1145/1807167.1807184.

Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. MIT Press, 1999.

James R. Norris. *Markov Chains*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 1998.

Judea Pearl. *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

Jian Pei, Jiawei Han, Behzad Mortazavi-Asl, Helen Pinto, Qiming Chen, Umeshwar Dayal, and Mei-Chun Hsu. PrefixSpan: Mining sequential patterns efficiently by prefix-projected pattern growth. In *Proceedings of the IEEE International Conference on Data Engineering*, pages 215–224, 2001. doi: 10.1109/ICDE.2001.914830.

Lawrence R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989. doi: 10.1109/5.18626.

Paul Resnick and Hal R. Varian. Recommender systems. *Communications of the ACM*, 40(3): 56–58, 1997. doi: 10.1145/245108.245121.

Retail Rocket. Retailrocket recommender system dataset. Kaggle, 2015. URL `https://www.kaggle.com/datasets/retailrocket/ecommerce-dataset`.

Saurabh Sahu, Amine Mhedhbi, Semih Salihoglu, Jimmy Lin, and M. Tamer Özsu. The ubiquity of large graphs and surprising challenges of graph processing. *Proceedings of the VLDB Endowment*, 11(4):420–431, 2017. doi: 10.14778/3166054.3166062.

Andrew J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967. doi: 10.1109/TIT.1967.1054010.

APPENDIX A: SOURCE CODE AVAILABILITY

The implementation of the *Customer Journey Path Optimization* framework was developed in **Python 3.13**. The complete source code is publicly available at:

https://github.com/The-Two-Y-s/Customer-Journey-Path-Optimization

The repository is organized into the following components:

- **Core algorithms** (`src/dijkstra.py`) — baseline Dijkstra with lazy deletion and the probability-pruned variant; both return structured results containing optimal path, log-cost, path probability, and four performance metrics (nodes explored, edges examined, edges relaxed, max priority-queue size).
- **Graph construction** (`src/graph_builder.py`) — builds a directed weighted graph from preprocessed transition data using an adjacency-list representation.
- **Preprocessing** (`src/preprocessing.py`) — ingests raw clickstream CSV data and computes maximum-likelihood transition probabilities.
- **Critical-τ analysis** (`src/critical_tau.py`) — adaptive fraction-based sweep to identify the largest τ^* preserving path optimality.
- **Synthetic generators** (`data/graph_generator.py`) — Erdős–Rényi and layered stage-based graphs with BFS connectivity guarantees and deterministic `hashlib.md5` seeding.
- **Real-data loaders** (`data/real_data_loader.py`) — parsers for RetailRocket (event-level and item-level) and RecSys 2015 YOOCHOOSE datasets.
- **Experiment runners** (`run_experiments.py`, `run_real_experiments.py`) — execute the full parameter matrix (2,160 synthetic + 840 real-data configurations).
- **Analysis notebook** (`analysis.ipynb`) — generates all plots and statistical tests (Wilcoxon signed-rank) from experiment CSV files.

Installation instructions, reproduction commands, and usage details are documented in the repository README.

Live Interactive Dashboard

An interactive dashboard visualising the experimental results and algorithm behaviour is publicly accessible at:

https://the-two-y-customer-journey-path-optimization.vercel.app/

The dashboard is built with React 18 and Vite, deployed on Vercel, and auto-redeploys on every push to the `main` branch. It presents speedup vs. τ curves, optimality gap analysis, scalability plots, the critical τ^* heatmap, and KPI summary cards derived from the experiment CSV files.